arXiv:2112.07464v1 [math.OC] 14 Dec 2021

Efficient differentiable quadratic programming layers: an ADMM approach

Andrew Butler and Roy H. Kwon
University of Toronto
Department of Mechanical and Industrial Engineering

December 15, 2021

Abstract

Recent advances in neural-network architecture allow for seamless integration of convex optimization problems as differentiable layers in an end-to-end trainable neural network. Integrating medium and large scale quadratic programs into a deep neural network architecture, however, is challenging as solving quadratic programs exactly by interior-point methods has worst-case cubic complexity in the number of variables. In this paper, we present an alternative network layer architecture based on the alternating direction method of multipliers (ADMM) that is capable of scaling to problems with a moderately large number of variables. Backward differentiation is performed by implicit differentiation of the residual map of a modified fixed-point iteration. Simulated results demonstrate the computational advantage of the ADMM layer, which for medium scaled problems is approximately an order of magnitude faster than the OptNet quadratic programming layer. Furthermore, our novel backward-pass routine is efficient, from both a memory and computation standpoint, in comparison to the standard approach based on unrolled differentiation or implicit differentiation of the KKT optimality conditions. We conclude with examples from portfolio optimization in the integrated prediction and optimization paradigm.

Keywords: Data driven stochastic-programming, differentiable neural networks, quadratic programming, ADMM

1 Introduction

Many problems in engineering, statistics and machine learning require solving convex optimization programs. In many real-world applications, the solution to the convex optimization program is a single component in a larger decision-making process (see for example [14, 13, 19]). Recent advances in neural-network architecture allow for seamless integration of convex optimization programs as differentiable layers in an end-to-end trainable neural network [1, 2, 3].

In this paper, we consider convex programming layers that take the form of parametric quadratic programs (PQPs) with linear equality and box inequality constraints:

$$\begin{aligned} \underset{\mathbf{z}}{\text{minimize}} \quad & \frac{1}{2}\,\mathbf{z}^T\,\mathbf{Q}(\boldsymbol{\theta})\,\mathbf{z} + \mathbf{z}^T\,\mathbf{p}(\boldsymbol{\theta}) \\ \text{subject to} \quad & \mathbf{A}(\boldsymbol{\theta})\,\mathbf{z} = \mathbf{b}(\boldsymbol{\theta}), \quad \mathbf{l}(\boldsymbol{\theta}) \leq \mathbf{z} \leq \mathbf{u}(\boldsymbol{\theta}) \end{aligned} \tag{1}$$

Here $\mathbf{z} \in \mathbb{R}^{d_z}$ denotes the decision variable and $\mathbf{Q}(\boldsymbol{\theta})$, $\mathbf{p}(\boldsymbol{\theta})$, $\mathbf{A}(\boldsymbol{\theta})$, $\mathbf{b}(\boldsymbol{\theta})$, $\mathbf{l}(\boldsymbol{\theta})$, $\mathbf{u}(\boldsymbol{\theta})$ are the parameterized problem variables. Program (1) occurs in many applications to statistics [41, 42], machine-learning [25, 29], signal-processing [30] and finance [8, 35, 34].

In general, a differentiable convex optimization layer can be viewed as a function that maps the program input variables to optimal primal(-dual) solution(s). Therefore, in a fully integrated system, optimizing problem variables by backpropogation ultimately requires computing the action of the Jacobian of the optimal solution(s) with respect to the corresponding program input variables. For example, the OptNet layer, proposed by Amos and Kolter [3] is a specialized differentiable optimization layer that uses a primal-dual interior-point method for small scale batch quadratic programs. The authors demonstrate that the solution to the system of equations provided by the KKT conditions at optimality provide a system of equations for implicit differentiation with respect to all relevant problem variables. Therefore, by strategically caching the KKT matrix factorization(s) (obtained during the forward-pass), then the gradients required for backpropogation are obtained at little extra additional cost. Indeed, the authors note that the OptNet layer is computationally efficient and therefore practical for small-scale problems ($d_z < 100$). However, solving convex quadratic programs exactly by interior-point methods has worst-case time complexity on the order of $\mathcal{O}(d_z^3)$ [27]. Therefore, for medium ($100 < d_z < 1000$) and large ($d_z > 1000$) scale problems, embedding an OptNet layer in a larger neural network can be computationally intractable.

In this paper, we address the computational challenges in the medium to large scale limit and propose an alternative differentiable network architecture for batch constrained quadratic programs of the form of Program (1). Our differentiable quadratic programming layer is built on top of the alternating direction method of multipliers (ADMM) algorithm, which recently has become increasingly popular for solving large-scale convex optimization problems [11, 36, 39, 40, 38]. Indeed, embedding the ADMM algorithm in a larger neural-network system has been fundamental to recent innovations in signal processing, compressed sensing, imaging and statistics (see for example [16, 45, 46]). The standard ADMM-network implementation computes the relevant gradients by unrolling the ADMM computational graph, which is memory inefficient and typically requires substantially larger networks. Furthermore, when the number of ADMM iterations is large in the forward-pass then the unrolled gradient can be computationally demanding. Alternatively, many differentiable convex optimization layers compute the relevant gradients by implicitly differentiating the Karush–Kuhn–Tucker (KKT) optimality conditions. Indeed differentiating the KKT system of equations is possible but unfortunately requires solving a system of equations of dimension $\mathbb{R}^{3d_z \times 3d_z}$, which can also be computationally impractical. As an alternative, we present a novel modified backward-pass routine that is efficient from both a memory and computational standpoint. Specifically, we recast the ADMM algorithm as a fixed-point iteration and apply the implicit function theorem to the resulting residual map in order to obtain the relevant backpropogation gradients. A primary advantage of our fixed-point differentiation is that the fixed-point iteration is of dimension d_z and therefore the resulting system of equations is approximately 3 times smaller than the equivalent KKT system. Finally, our differentiable ADMM-layer and all algorithmic implementations is made available as an open-source R package, available here:

https://github.com/adsb85/lqp

The remainder of the paper is outlined as follows. We begin with the problem motivation and a brief discussion of related work in the field of differentiable convex optimization layers. In Section 2

we review the ADMM algorithm and present our ADMM-based architecture for forward solving batch PQPs. We review the KKT based implicit differentiation and then present the fixed-point iteration and derive the expression for the relevant gradients. In Section 3 we perform several simulations (under numerous model specifications) and compare the computational efficiency and performance accuracy of our ADMM-layer with the state-of-the-art OptNet layer. We demonstrate that for medium-scale problems, our ADMM-layer implementation is approximately an order of magnitude faster than the OptNet layer and provides solutions that are equally as optimal. Moreover, we compare the computational efficiency of the backpropogation routines based on unrolled differentiation, KKT implicit differentiation and fixed-point implicit differentiation. We demonstrate that the fixed-point implicit differentiation is universally more efficient than the KKT implicit differentiation, and under certain conditions is preferred to the unrolled differentiation. We conclude with a real world application of the ADMM-layer to a medium scale portfolio optimization problem in the integrated prediction and optimization paradigm.

1.1 Related work:

1.1.1 Problem Motivation

Many problems in machine learning, statistics, engineering and operations research involve both predictive forecasting and decision based optimization. Recent advances in neural network architecture embed convex optimization programs as differentiable layers in a larger neural network structure. A fully integrated prediction and optimization (IPO) architecture therefore enables the integration of predictive forecasting and convex optimization and allows for the minimization of the total decision error induced by the forecast estimates (see for example [13, 14, 19, 20, 21]). This is in contrast to a more traditional 'predict, then optimize' approach which would first fit the predictive models (for example by maximum likelihood or least-squares) and then 'plug-in' those predictions to the corresponding decision-based optimization program. While it is true that a perfect predictive model would lead to optimal decision making, in reality, all predictive models do make some error, and thus an inefficiency exists in the 'predict, then optimize' paradigm. With the widespread adoption of machine-learning and data science in operations research, there has been a growing body of literature on data-driven decision making and the relative merits of decoupled versus integrated predictive decision-making [4, 7, 28, 33, 32, 43].

The preliminary findings of the aforementioned work advocate strongly for an IPO approach. Indeed, IPO models typically exhibit lower model complexity and demonstrate statistically significant improvements in out-of-sample performance in comparison to more traditional 'predict, then optimize' models. Unfortunately, training medium and large scale IPO models can be computationally demanding. For example, Elmachtoub and Grigas [20], Elmachtoub et al. [21] report that their 'smart predict, then optimize' models can take several hours to train medium and large scale problems, while traditional prediction methods typically take seconds or minutes to train. Butler and Kwon [13] provide details on the computational complexity of IPO models trained with the OptNet layer and demonstrate that when $d_z > 100$ then fitting IPO models can be computationally impractical. Improving the efficiency of the integrated framework is therefore an open and important area of research.

1.1.2 Differentiable convex optimization layers:

Differentiable convex optimization layers provide an efficient and seamless framework for integrating predictive forecasting with downstream decision-based optimization in a end-to-end trainable neural network. Modern neural network technology (such as *torch* or *tensorflow*) require that every layer in the network inherits a forward and backward-pass routine. For differentiable convex optimization layers , the forward-pass is typically an iterative optimization algorithm that converts problem variables into optimal primal(-dual) solution(s). The backward-pass routine therefore computes the action of the Jacobian of the optimal solution(s) with respect to all problem variables, and returns the left matrix-vector product of the Jacobian with the previous backward-pass gradient(s).

For example, the state-of-the-art OptNet layer implements a primal-dual interior-point method for solving small-scale batch constrained quadratic programs [3]. For backward differentiation, the authors implement a novel and efficient argmin differentiation routine that implicitly differentiates the KKT system of equations at optimality. By strategically caching the factorized KKT left-hand side matrix then the resulting method is shown to be computationally tractable for small problems within the context of deep neural network architectures. The author's acknowledge, however, that the OptNet layer may be impractical for optimization problems with a moderate to large number of variables.

More recently, Agrawal et al. [2] provide a general framework for differentiable convex cone programming. Their forward-pass recasts the conic program in its equivalent homogeneous self-dual embedding form, which is then solved by operator splitting [36]. In the backward-pass, the relevant gradients are obtained by implicit differentiation of the residual map provided by the homogeneous self-dual embedding. The resulting differentiable cone programming layer is flexible, but requires the user to transform their problem into a canonical form, which is often time-consuming, prone to error and requires a certain level of domain expertise.

Alternatively, Agrawal et al. [1], provide a domain-specific language for differentiable disciplined convex programs. Their approach abstracts away the process of converting problems to canonical form with minimal loss in computational efficiency in comparison to specialized convex optimization layers. They also provide an efficient sparse matrix solver, which for sparse quadratic programs is on average an order of magnitude faster than the OptNet layer. Similarly, Blondel et al. [9] provide an efficient and modular approach for implicit differentiation of optimization problems. They consider KKT, proximal gradient and mirror descent fixed-point implicit differentiation and provide a software infrastructure for efficiently integrating their modular implicit differentiation routines with state-of-the-art optimization solvers. That said, for solving batches of convex optimization problems it is often preferred and more efficient to avail of optimization solvers that have the ability to exploit fast GPU-based batch solves.

1.1.3 ADMM and unrolled differentiation:

The alternating direction method of multipliers (ADMM) algorithm, first proposed by Gabay and Mercier [24] and Glowinski and Marroco [26], is well suited to many large-scale and distributed problems common to applications of statistics, machine learning, control and finance. We note that the ADMM algorithm is closely related to algorithms such as dual ascent, the augmented Lagrangian method of multipliers, and operator (Douglas–Rachford) splitting and refer to Boyd et al. [11] for a

comprehensive overview.

Embedding the above mentioned algorithms in larger neural-network structures has been fundamental to recent innovations in signal processing, compressed sensing, imaging and statistics (see for example [6, 16, 17, 23, 31]). Perhaps most closely related to our own work, the ADMM-Net, first proposed by Yang et al. [46], recasts and embeds the iterative ADMM procedure as a fully learnable network graph. The authors provide examples from compressive sensing magnetic resonance imaging and demonstrate that their ADMM-Net achieves state-of-the-art model accuracy and computationally efficiency. More recently, the Differentiable Linearized ADMM (D-LADMM) algorithm, proposed by [45], generalizes the ADMM-Net and is capable of solving general deep learning problems with equality constraints. The authors show that there exists a set of learnable parameters for D-LADMM to generate global solutions and provide the relevant convergence analysis. However, in all cases mentioned above the authors consider either unconstrained or linear equality constrained least-squares problems and do not consider inequality constraints. Furthermore, they perform the action of argmin differentiation by unrolling the 'inner-loop' of the convex optimization routine, which necessitates substantially larger and less efficient networks [3, 4].

Our ADMM-layer derives inspiration from both the differentiable convex optimization and ADMM network literature. To our knowledge, our ADMM-layer is the first implementation of its kind that can efficiently handle medium to large scale differentiable constrained PQPs. In this paper we demonstrate that solving medium and large scale QPs by interior-point methods can be computationally burdensome. An obvious course of action is to replace the interior-point algorithm in the forward-pass with a more computationally efficient first-order method, such as ADMM. However, implementing an efficient backward-pass routine by implicit differentiation of the KKT conditions is challenging as the ADMM algorithm does not explicit solve the KKT system of equations. Therefore, unlike the OptNet implementation, at each 'outer' iteration (hereafter referred to as epochs) we must form and solve the resulting KKT system, which for large scale problems creates a computational bottleneck. Unrolled differentiation of the ADMM algorithm is appropriate for small scale problems. However, for larger scale problems or for problems that require solving the convex optimization problem to a high degree of accuracy, an unrolled differentiation approach can also be computationally impractical. In contrast, our novel fixed-point implicit differentiation method is shown to be computationally efficient and invariant to the number of 'inner' iterations performed in the ADMM forward-pass. Furthermore, as mentioned earlier, a primary advantage of the fixed-point differentiation is that the fixed-point iterative scheme is of dimension d_z and is therefore approximately 3 times smaller than the KKT system. We demonstrate that in the absence of a pre-factorized KKT system, the fixed-point implicit differentiation is preferred to the KKT implicit differentiation and is competitive with the efficient KKT factorization caching provided in the OptNet layer. Furthermore, we demonstrate that for small-scale problems, our ADMM-layer is competitive with the state-of-the-art OptNet layer in terms of accuracy and computational efficiency. For medium and large scale problems, our ADMM layer is shown to be approximately an order of magnitude faster than the OptNet layer. Of course, our ADMM-layer is not without its own limitations and areas for improvement, discussed in detail in Section 4.

2 Methodology

In general, the ADMM algorithm is applied to problems of the form:

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{subject to} \quad & \mathbf{A}\,\mathbf{x} + \mathbf{B}\,\mathbf{z} = \mathbf{c} \end{aligned} \tag{2}$$

with decision variables $\mathbf{x} \in \mathbb{R}^{d_x}$, $\mathbf{z} \in \mathbb{R}^{d_z}$ and problem variables $\mathbf{A} \in \mathbb{R}^{d_{eq} \times d_x}$, $\mathbf{B} \in \mathbb{R}^{d_{eq} \times d_z}$ and $\mathbf{c} \in \mathbb{R}^{d_{eq}}$. In order to guarantee convergence we assume that $f\colon \mathbb{R}^{d_x} \to \mathbb{R}$ and $g\colon \mathbb{R}^{d_z} \to \mathbb{R}$ are closed, proper convex functions [11]. The augmented Lagrangian of Program (2) is given by:

$$L_\rho(\mathbf{x}, \mathbf{z}, \mathbf{y}) = f(\mathbf{x}) + g(\mathbf{z}) + \boldsymbol{\lambda}^T(\mathbf{r}) + \frac{\rho}{2}\|\mathbf{r}\|_2^2, \tag{3}$$

with Lagrange dual variable $\boldsymbol{\lambda}$, residual $\mathbf{r} = \mathbf{A}\,\mathbf{x} + \mathbf{B}\,\mathbf{z} - \mathbf{c}$ and user-defined penalty parameter $\rho > 0$. We denote $\boldsymbol{\mu} = \rho^{-1}\,\boldsymbol{\lambda}$ and therefore the well-known scaled ADMM iterations are as follows:

$$\begin{aligned} \mathbf{x}^{k+1} &= \operatorname*{argmin}_{\mathbf{x}} f(\mathbf{x}) + \frac{\rho}{2}\|\mathbf{A}\,\mathbf{x} + \mathbf{B}\,\mathbf{z}^k - \mathbf{c} + \boldsymbol{\mu}^k\|_2^2 \\ \mathbf{z}^{k+1} &= \operatorname*{argmin}_{\mathbf{z}} g(\mathbf{z}) + \frac{\rho}{2}\|\mathbf{A}\,\mathbf{x}^{k+1} + \mathbf{B}\,\mathbf{z} - \mathbf{c} + \boldsymbol{\mu}^k\|_2^2 \\ \boldsymbol{\mu}^{k+1} &= \boldsymbol{\mu}^k + \mathbf{A}\,\mathbf{x}^{k+1} + \mathbf{B}\,\mathbf{z}^{k+1} - \mathbf{c} \end{aligned} \tag{4}$$

where $\mathbf{x}^k$ and $\mathbf{z}^k$ denote the decision variables at iteration k.

We denote $\mathbf{r}^k = \mathbf{A}\,\mathbf{x}^k + \mathbf{B}\,\mathbf{z}^k - \mathbf{c}$ and $\mathbf{s}^k = \rho\,\mathbf{A}^T\,\mathbf{B}(\mathbf{z}^k - \mathbf{z}^{k-1})$ as the primal and dual residual at iteration k. Let $\epsilon^p > 0$ and $\epsilon^d > 0$ be the user defined stopping tolerances for the primal and dual residuals, respectively. Therefore a reasonable stopping criteria would be:

$$\mathbf{r}^k \leq \epsilon^p \quad \text{and} \quad \mathbf{s}^k \leq \epsilon^d. \tag{5}$$

2.1 ADMM for parametric quadratic programs

We consider convex parametric quadratic programs (PQPs) of the form:

$$\begin{aligned} \underset{\mathbf{z}}{\text{minimize}} \quad & \frac{1}{2}\,\mathbf{z}^T\,\mathbf{Q}(\boldsymbol{\theta})\,\mathbf{z} + \mathbf{z}^T\,\mathbf{p}(\boldsymbol{\theta}) \\ \text{subject to} \quad & \mathbf{A}(\boldsymbol{\theta})\,\mathbf{z} = \mathbf{b}(\boldsymbol{\theta}), \quad \mathbf{l}(\boldsymbol{\theta}) \leq \mathbf{z} \leq \mathbf{u}(\boldsymbol{\theta}), \end{aligned} \tag{6}$$

with decision variable $\mathbf{z} \in \mathbb{R}^{d_z}$. The objective function is therefore defined by a vector $\mathbf{p}(\boldsymbol{\theta}) \in \mathbb{R}^{d_z}$ and symmetric positive definite matrix $\mathbf{Q}(\boldsymbol{\theta}) \in \mathbb{R}^{d_z \times d_x}$. Here, $\mathbf{A}(\boldsymbol{\theta}) \in \mathbb{R}^{d_{eq} \times d_z}$, $\mathbf{b}(\boldsymbol{\theta}) \in \mathbb{R}^{d_{eq}}$, $\mathbf{l}(\boldsymbol{\theta}) \in \mathbb{R}^{d_z}$ and $\mathbf{u}(\boldsymbol{\theta}) \in \mathbb{R}^{d_z}$ define the linear equality and box inequality constraints. We assume that all problem variables are parameterized by $\boldsymbol{\theta}$ and are therefore trainable when integrated in an end-to-end neural network; rather than simply being supplied by the user.

2.1.1 ADMM-layer: forward-pass

Our ADMM-layer solves Program (6) in the forward-pass by applying the ADMM algorithm as outlined in Section 2. Note that for ease of notation we temporarily drop the parameterization, $\boldsymbol{\theta}$.

We define

$$f(\mathbf{x}) = \frac{1}{2}\,\mathbf{x}^T\,\mathbf{Q}\,\mathbf{x} + \mathbf{x}^T\,\mathbf{p}, \tag{7}$$

with domain $\{\mathbf{x} \,|\, \mathbf{A}\,\mathbf{x} = \mathbf{b}\}$. Similarly, we define

$$g(\mathbf{z}) = \mathbb{I}_{\mathbf{l} \leq \mathbf{z} \leq \mathbf{u}}(\mathbf{z}) \tag{8}$$

where $\mathbb{I}_{\mathbf{l} \leq \mathbf{z} \leq \mathbf{u}}(\mathbf{z})$ denotes the indicator function with respect to the linear inequality constraints. Program (6) is then recast to the following convex optimization Program:

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{subject to} \quad & \mathbf{x} - \mathbf{z} = 0. \end{aligned} \tag{9}$$

Applying the ADMM iterations, as defined by Equations (4), to Program (9) therefore gives the following iterative optimization algorithm:

$$\mathbf{x}^{k+1} = \underset{\{\mathbf{x} \,|\, \mathbf{A}\,\mathbf{x} = \mathbf{b}\}}{\operatorname{argmin}} \frac{1}{2}\,\mathbf{x}^T\,\mathbf{Q}\,\mathbf{x} + \mathbf{x}^T\,\mathbf{p} + \frac{\rho}{2}\|\mathbf{x} - \mathbf{z}^k + \boldsymbol{\mu}^k\|_2^2 \tag{10a}$$

$$\mathbf{z}^{k+1} = \underset{\{\mathbf{l} \leq \mathbf{z} \leq \mathbf{u}\}}{\operatorname{argmin}} \frac{\rho}{2}\|\mathbf{x}^{k+1} - \mathbf{z} + \boldsymbol{\mu}^k\|_2^2 \tag{10b}$$

$$\boldsymbol{\mu}^{k+1} = \boldsymbol{\mu}^k + \mathbf{x}^{k+1} - \mathbf{z}^{k+1} \tag{10c}$$

The ADMM algorithm, as defined by Equations (10), allows for efficient optimization of medium and large scale quadratic programs. Firstly, we note that (10b) is a least squares problem with box-inequality constraints, and therefore can be solved analytically. Specifically, we define the euclidean projection onto a set of box constraints as:

$$\Pi(\mathbf{x}) = \begin{cases} \mathbf{l}_j & \text{if } \mathbf{x}_j < \mathbf{l}_j \\ \mathbf{x}_j & \text{if } \mathbf{l}_j \leq \mathbf{x}_j \leq \mathbf{l}_j \\ \mathbf{u}_j & \text{if } \mathbf{x}_j > \mathbf{u}_j \end{cases}. \tag{11}$$

The analytic solution to Program (10b) is therefore given by:

$$\mathbf{z}^{k+1} = \Pi(\mathbf{x}^{k+1} + \boldsymbol{\mu}^k). \tag{12}$$

Furthermore, Program (10a) is an equality constrained quadratic program, which can also be solved analytically. Specifically, the KKT optimality conditions of Program (10a) can be expressed as the solution to the following linear system of equations:

$$\begin{bmatrix} \mathbf{Q} + \rho\,\mathbf{I}_{\mathbf{x}} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \boldsymbol{\eta}^{k+1} \end{bmatrix} = - \begin{bmatrix} \mathbf{p} - \rho(\mathbf{z}^k - \boldsymbol{\mu}^k) \\ -\mathbf{b} \end{bmatrix}, \tag{13}$$

with identity matrix $\mathbf{I_x} \in \mathbb{R}^{d_x \times d_x}$. Applying Equations (12) and (13) allows us to express the ADMM iterations in a simplified form:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \boldsymbol{\eta}^{k+1} \end{bmatrix} = - \begin{bmatrix} \mathbf{Q} + \rho \mathbf{I_x} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{p} - \rho(\mathbf{z}^k - \boldsymbol{\mu}^k) \\ -\mathbf{b} \end{bmatrix} \tag{14a}$$

$$\mathbf{z}^{k+1} = \Pi(\mathbf{x}^{k+1} + \boldsymbol{\mu}^k) \tag{14b}$$

$$\boldsymbol{\mu}^{k+1} = \boldsymbol{\mu}^k + \mathbf{x}^{k+1} - \mathbf{z}^{k+1} \tag{14c}$$

Observe that the per-iteration cost of the ADMM algorithm is dominated by solving the system of Equations (13). Note, however, that this linear system is in general smaller than the Newton system found in a standard primal-dual interior-point solvers by a factor of approximately 5, and therefore remains tractable for medium and large scale problems. Furthermore, if ρ is static then the left-hand side matrix in Equation (13) remains unchanged at each iteration and therefore is factorized only once at the onset of the ADMM algorithm. Furthermore, as we demonstrate below, this matrix factorization can subsequently be cached and invoked during backpropogation to compute the relevant gradients. Lastly, we note that if the matrices $\mathbf{Q}$ and $\mathbf{A}$ remain unchanged at each epoch of gradient descent, then the left-hand-side matrix needs to only be factorized **once** during the entire training process.

2.1.2 ADMM-layer: unrolled differentiation

Note that the standard unrolled differentiation approach 'unrolls' the iterations in Equation (14) by standard backpropogation [37] and requires that each operation in Equation (14) be differentiable. We refer to Domke [17] and Diamond et al. [16] for a more comprehensive overview of unrolled differentiation. Our unrolled differentiation is invoked by the standard auto-differentiation routine in the *torch* library.

2.1.3 ADMM-layer: KKT implicit differentiation

As an alternative to unrolled differentiation, we note that the system of equations provided by the KKT conditions at optimality is a fixed point mapping. As outlined by [3, 4], it is therefore possible to apply the implicit function theorem and derive the gradient of the primal-dual variables with respect to the problem variables in Program (6). In this section we derive the KKT implicit differentiation for our ADMM solver. We begin with a few definitions.

Definition 1. *Let* $F \colon \mathbb{R}^{d_v} \times \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_v}$ *be a continuously differentiable function with variable* $\mathbf{v}$ *and parameter* $\boldsymbol{\theta}$*. We define* $\mathbf{v}^*$ *as a* **fixed-point** *of* F *at* $(\mathbf{v}^*, \boldsymbol{\theta})$ *if:*

$$F(\mathbf{v}^*, \boldsymbol{\theta}) = \mathbf{v}^* \,.$$

Definition 2. *The* **residual map**, $G \colon \mathbb{R}^{d_v} \times \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_v}$ *of a fixed point,* $(\mathbf{v}^*, \boldsymbol{\theta})$*, of* F *is given by:*

$$G(\mathbf{v}^*, \boldsymbol{\theta}) = F(\mathbf{v}^*, \boldsymbol{\theta}) - \mathbf{v}^* = 0.$$

The **implicit function theorem**, as defined by Dontchev and Rockafellar [18], then provides the conditions on G for which the Jacobian of the solution mapping with respect to $\boldsymbol{\theta}$ is well defined.

Theorem 1. *Let $G\colon \mathbb{R}^{d_v} \times \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_v}$ be a continuously differentiable function in a neighborhood of $(\mathbf{v}^*, \boldsymbol{\theta})$ such that $G(\mathbf{v}^*, \boldsymbol{\theta}) = 0$. Denote the nonsingular partial Jacobian of G with respect to $\mathbf{v}$ as $\nabla_{\mathbf{v}} G(\mathbf{v}^*, \boldsymbol{\theta})$. Then $\mathbf{v}(\boldsymbol{\theta})$ is an implicit function of $\boldsymbol{\theta}$ and is continuously differentiable in a neighborhood, Θ, of $\boldsymbol{\theta}$ with Jacobian:*

$$\nabla_{\boldsymbol{\theta}}\, \mathbf{v}(\boldsymbol{\theta}) = -[\nabla_{\mathbf{v}} G(\mathbf{v}(\boldsymbol{\theta}), \boldsymbol{\theta})]^{-1} \nabla_{\boldsymbol{\theta}} G(\mathbf{v}(\boldsymbol{\theta}), \boldsymbol{\theta}) \quad \forall \quad \boldsymbol{\theta} \in \Theta. \tag{15}$$

Corollary 1. *Let $F\colon \mathbb{R}^{d_v} \times \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_v}$ be a continuously differentiable function with fixed-point $(\mathbf{v}^*, \boldsymbol{\theta})$. Then $\mathbf{v}(\boldsymbol{\theta})$ is an implicit function of $\boldsymbol{\theta}$ and is continuously differentiable in a neighborhood, Θ, of $\boldsymbol{\theta}$ with Jacobian:*

$$\nabla_{\boldsymbol{\theta}}\, \mathbf{v}(\boldsymbol{\theta}) = [\mathbf{I}_{\mathbf{v}} - \nabla_{\mathbf{v}} F(\mathbf{v}(\boldsymbol{\theta}), \boldsymbol{\theta})]^{-1} \nabla_{\boldsymbol{\theta}} F(\mathbf{v}(\boldsymbol{\theta}), \boldsymbol{\theta}) \quad \forall \quad \boldsymbol{\theta} \in \Theta. \tag{16}$$

For constrained quadratic programming, let us denote the primal-dual solution at optimality by $\boldsymbol{\nu}^* = (\mathbf{z}^*, \tilde{\boldsymbol{\lambda}}^*, \boldsymbol{\eta}^*)$, where $\mathbf{z}^*$ and $\boldsymbol{\eta}^*$ are defined by Equations (13) at optimality. Note that the dual variables associated with the inequality constraints are given by $\tilde{\boldsymbol{\lambda}}^* = (\boldsymbol{\lambda}_-^*, \boldsymbol{\lambda}_+^*)$ with:

$$\boldsymbol{\lambda}_-^* = -\min(\rho\, \boldsymbol{\mu}^*, 0) \quad \text{and} \quad \boldsymbol{\lambda}_+^* = \max(\rho\, \boldsymbol{\mu}^*, 0). \tag{17}$$

We define the box inequality constraints as $\mathbf{G}\,\mathbf{z} \leq \mathbf{h}$, where

$$\mathbf{G} = \begin{bmatrix} -\mathbf{I}_{\mathbf{x}} \\ \mathbf{I}_{\mathbf{x}} \end{bmatrix} \quad \text{and} \quad \mathbf{h} = \begin{bmatrix} -\mathbf{l} \\ \mathbf{u} \end{bmatrix}.$$

Note that all constraints are affine and therefore Slater's condition reduces to feasibility. The KKT conditions for stationarity, primal feasibility, and complementary slackness therefore defines a fixed-point at optimality $\boldsymbol{\nu}^*$ given by:

$$G(\boldsymbol{\nu}^*, \boldsymbol{\theta}) = \begin{bmatrix} \mathbf{p} + \mathbf{Q}\,\mathbf{z}^* + \mathbf{G}^T \tilde{\boldsymbol{\lambda}}^* + \mathbf{A}^T \boldsymbol{\eta}^* \\ \operatorname{diag}(\tilde{\boldsymbol{\lambda}}^*)(\mathbf{G}\,\mathbf{z}^* - \mathbf{h}) \\ \mathbf{A}\,\mathbf{z}^* - \mathbf{b} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} \tag{18}$$

Applying Theorem 1, we take the differential of these conditions to give the following system of equations:

$$\begin{bmatrix} \mathbf{Q} & \mathbf{G}^T & \mathbf{A}^T \\ \operatorname{diag}(\tilde{\boldsymbol{\lambda}}^*)\,\mathbf{G} & \operatorname{diag}(\mathbf{G}\,\mathbf{z}^* - \mathbf{h}) & 0 \\ \mathbf{A} & 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{d}\mathbf{z} \\ \mathbf{d}\,\boldsymbol{\lambda} \\ \mathbf{d}\,\boldsymbol{\nu} \end{bmatrix} = - \begin{bmatrix} \mathbf{d}\mathbf{Q}\,\mathbf{z}^* + \mathbf{d}\mathbf{p} + \mathbf{d}\mathbf{G}^T \tilde{\boldsymbol{\lambda}}^* + \mathbf{d}\mathbf{A}^T\, \boldsymbol{\eta} * \\ \operatorname{diag}(\tilde{\boldsymbol{\lambda}}^*)\mathbf{d}\mathbf{G}\,\mathbf{z}^* - \operatorname{diag}(\tilde{\boldsymbol{\lambda}}^*)\mathbf{d}\mathbf{h} \\ \mathbf{d}\mathbf{A}\,\mathbf{z}^* - \mathbf{d}\mathbf{b} \end{bmatrix}. \tag{19}$$

Observe that the left side matrix gives the optimality conditions of the convex quadratic problem, which, when solving by interior-point methods, must be factorized in order to obtain the solution to the nominal program [10]. The right side gives the differentials of the relevant functions at the achieved solution with respect to any of the problem variables. In practice, however, we never explicitly form the right-side Jacobian matrix directly. Instead we follow the work of Amos and Kolter [3] and compute the left matrix-vector product of the Jacobian with the previous backward-pass gradient, $\frac{\partial \ell}{\partial \mathbf{z}^*}$, as outlined below:

$$\begin{bmatrix} \bar{\mathbf{d}}_{\mathbf{z}} \\ \bar{\mathbf{d}}_{\boldsymbol{\lambda}} \\ \bar{\mathbf{d}}_{\boldsymbol{\eta}} \end{bmatrix} = - \begin{bmatrix} \mathbf{Q} & \mathbf{G}^T \operatorname{diag}(\tilde{\boldsymbol{\lambda}}^*) & \mathbf{A}^T \\ \mathbf{G} & \operatorname{diag}(\mathbf{G}\,\mathbf{z}^* - \mathbf{h}) & 0 \\ \mathbf{A} & 0 & 0 \end{bmatrix}^{-1} \begin{bmatrix} \left(\frac{\partial \ell}{\partial \mathbf{z}^*}\right)^T \\ 0 \\ 0 \end{bmatrix}. \tag{20}$$

Equation (20) allows for efficient computation of the gradients with respect to any of the relevant problem variables. This is particularly true when using interior-point methods as the required gradients are effectively obtained 'for free' upon factorization of the left matrix when obtaining the solution, $\mathbf{z}^*$, in the forward-pass. In the ADMM algorithm, however, we never explicitly solve the KKT system of equations and therefore we must form and factorize the left side KKT matrix during the backward pass routine. Observe that the left-side matrix in Equation (20) is of dimension $3d_z + d_{eq}$ and therefore for large scale problems solving this system of equations can be computationally burdensome. Finally, for the reader's interest, we state the gradients for all problem variables and refer the reader to Amos and Kolter [3] for their derivation.

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{Q}} &= \frac{1}{2}\left(\bar{\mathbf{d}}_{\mathbf{z}}\,\mathbf{z}^{*T} + \mathbf{z}^*\,\bar{\mathbf{d}}_{\mathbf{z}}^T\right) & \frac{\partial \ell}{\partial \mathbf{p}} &= \bar{\mathbf{d}}_{\mathbf{z}} \\ \frac{\partial \ell}{\partial \mathbf{A}} &= \bar{\mathbf{d}}_{\boldsymbol{\eta}}\,\mathbf{z}^{*T} + \boldsymbol{\eta}^*\,\bar{\mathbf{d}}_{\mathbf{z}}^T & \frac{\partial \ell}{\partial \mathbf{b}} &= -\bar{\mathbf{d}}_{\boldsymbol{\eta}} \\ \frac{\partial \ell}{\partial \mathbf{G}} &= \operatorname{diag}(\boldsymbol{\lambda}^*)\bar{\mathbf{d}}_{\boldsymbol{\lambda}}\,\mathbf{z}^{*T} + \boldsymbol{\lambda}^*\,\bar{\mathbf{d}}_{\mathbf{z}}^T & \frac{\partial \ell}{\partial \mathbf{h}} &= -\operatorname{diag}(\boldsymbol{\lambda}^*)\bar{\mathbf{d}}_{\boldsymbol{\lambda}} \end{aligned} \tag{21}$$

2.1.4 ADMM-layer: fixed-point implicit differentiation

In this section we demonstrate that the ADMM iterations in Equation (13) can be cast as a fixed-point iteration of dimension $d_z + d_{eq}$. In many applications, d_{eq} is typically much smaller than d_z, and therefore the proposed fixed-point differentiation will almost certainly decrease the computational overhead of the backward-pass routine. We begin with the following proposition. Note that all proofs are available in the Appendix.

Proposition 1. *Let* $\mathbf{v}^k = \mathbf{x}^{k+1} + \boldsymbol{\mu}^k$, $\tilde{\mathbf{v}}^k = (\mathbf{v}^k, \boldsymbol{\eta}^k)$ *and define* $F\colon \mathbb{R}^{d_{\tilde{v}}} \times \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_{\tilde{v}}}$. *Then the ADMM iterations in Equation* (13) *can be cast as a fixed-point iteration of the form* $F(\tilde{\mathbf{v}}, \boldsymbol{\theta}) = \tilde{\mathbf{v}}$ *given by:*

$$\begin{bmatrix} \mathbf{v}^{k+1} \\ \boldsymbol{\eta}^{k+2} \end{bmatrix} = - \begin{bmatrix} \mathbf{Q} + \rho\,\mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{p} - \rho(2\Pi(\mathbf{v}^k) - \mathbf{v}^k) \\ -\mathbf{b} \end{bmatrix} + \begin{bmatrix} \mathbf{v}^k \\ \boldsymbol{\eta}^{k+1} \end{bmatrix} - \begin{bmatrix} \Pi(\mathbf{v}^k) \\ \boldsymbol{\eta}^{k+1} \end{bmatrix}. \tag{22}$$

We follow Busseti et al. [12] and define the derivative of the projection operator, Π as:

$$D\Pi(\mathbf{x}) = \begin{cases} 0 & \text{if } \mathbf{x}_j < \mathbf{l}_j \\ 1 & \text{if } \mathbf{l}_j \le \mathbf{x}_j \le \mathbf{l}_j \\ 0 & \text{if } \mathbf{x}_j > \mathbf{u}_j \end{cases}. \tag{23}$$

Observe that $D\Pi(\mathbf{x})$ is not continuously differentiable when $\mathbf{x}_j = \mathbf{l}_j$ or $\mathbf{x}_j = \mathbf{u}_j$. In practice, we can overcome the non-differentiability of Π by introducing a small perturbation to $\mathbf{x}$, thus moving $\mathbf{x}$

away from the boundaries. Alternatively, smooth sigmoid based approximations to $\Pi(\mathbf{x})$ may also be suitable. In all experiments below, however, we invoke $D\Pi(\mathbf{x})$ directly as defined in Equation (23).

The Jacobian, $\nabla_{\tilde{\mathbf{v}}} F$, is therefore defined as:

$$\nabla_{\tilde{\mathbf{v}}} F = -\begin{bmatrix} \mathbf{Q} + \rho\,\mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix}^{-1} \begin{bmatrix} -\rho(2D\Pi(\mathbf{v}) - \mathbf{I}_{\mathbf{v}}) & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} \mathbf{I}_{\mathbf{v}} & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} - \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix}. \tag{24}$$

Corollary 1 therefore gives the desired Jacobian, $\nabla_{\boldsymbol{\theta}} \tilde{\mathbf{v}}(\boldsymbol{\theta})$, with respect to the parameter $\boldsymbol{\theta}$:

$$\nabla_{\boldsymbol{\theta}} \tilde{\mathbf{v}}(\boldsymbol{\theta}) = [\mathbf{I}_{\tilde{\mathbf{v}}} - \nabla_{\tilde{\mathbf{v}}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta})]^{-1} \nabla_{\boldsymbol{\theta}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta}) \tag{25}$$

From the definition of $\mathbf{v}$ we have that the Jacobians $\nabla_{\boldsymbol{\theta}}\, \mathbf{x}(\boldsymbol{\theta})$ and $\nabla_{\boldsymbol{\theta}}\, \boldsymbol{\eta}(\boldsymbol{\theta})$ are given by:

$$\begin{bmatrix} \nabla_{\boldsymbol{\theta}}\, \mathbf{x}(\boldsymbol{\theta}) \\ \nabla_{\boldsymbol{\theta}}\, \boldsymbol{\eta}(\boldsymbol{\theta}) \end{bmatrix} = \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \Big[\mathbf{I}_{\tilde{\mathbf{v}}} - \nabla_{\tilde{\mathbf{v}}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta}) \Big]^{-1} \nabla_{\boldsymbol{\theta}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta}) \tag{26}$$

As before we never form the Jacobians $\nabla_{\boldsymbol{\theta}}\, \mathbf{x}(\boldsymbol{\theta})$ and $\nabla_{\boldsymbol{\theta}}\, \boldsymbol{\eta}(\boldsymbol{\theta})$ directly. Instead, we compute the left matrix-vector product of the Jacobian with the previous backward-pass gradient, $\frac{\partial \ell}{\partial \mathbf{z}^*}$, as outlined below.

Proposition 2. *Let $\hat{\mathbf{d}}_{\mathbf{x}}$ and $\hat{\mathbf{d}}_{\boldsymbol{\eta}}$ be defined as:*

$$\begin{aligned} \begin{bmatrix} \hat{\mathbf{d}}_{\mathbf{x}} \\ \hat{\mathbf{d}}_{\boldsymbol{\eta}} \end{bmatrix} &= \begin{bmatrix} \mathbf{Q} + \rho\,\mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \Big[\mathbf{I}_{\tilde{\mathbf{v}}} - \nabla_{\tilde{\mathbf{v}}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta}) \Big]^{-T} \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \begin{bmatrix} \big(-\frac{\partial \ell}{\partial \mathbf{z}^*} \big)^T \\ 0 \end{bmatrix} \\ &= \left[\begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \begin{bmatrix} \mathbf{Q} + \rho\,\mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix} + \begin{bmatrix} -\rho(2D\Pi(\mathbf{v}) - \mathbf{I}_{\mathbf{v}}) & 0 \\ 0 & 0 \end{bmatrix} \right]^{-1} \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \begin{bmatrix} \big(-\frac{\partial \ell}{\partial \mathbf{z}^*} \big)^T \\ 0 \end{bmatrix}. \end{aligned} \tag{27}$$

Then the gradients of the loss function, ℓ, with respect to problem variables $\mathbf{Q}$, $\mathbf{p}$, $\mathbf{A}$ and $\mathbf{b}$ are given by: .

$$\begin{aligned} \frac{\partial \ell}{\partial\, \mathbf{Q}} &= \frac{1}{2}\Big(\hat{\mathbf{d}}_{\mathbf{x}}\, \mathbf{x}^{*T} + \mathbf{x}^*\, \hat{\mathbf{d}}_{\mathbf{x}}^T \Big) & \frac{\partial \ell}{\partial\, \mathbf{p}} &= \hat{\mathbf{d}}_{\mathbf{x}} \\ \frac{\partial \ell}{\partial\, \mathbf{A}} &= \hat{\mathbf{d}}_{\boldsymbol{\eta}}\, \mathbf{x}^{*T} + \boldsymbol{\eta}^*\, \hat{\mathbf{d}}_{\mathbf{x}}^T & \frac{\partial \ell}{\partial\, \mathbf{b}} &= -\hat{\mathbf{d}}_{\boldsymbol{\eta}} \end{aligned} \tag{28}$$

Computing the gradients of the loss with respect to the box constraint variables, $\mathbf{l}$ and $\mathbf{u}$ is also straightforward.

Proposition 3. *Define $\tilde{\boldsymbol{\mu}}^*$ and $\hat{\mathbf{d}}_{\boldsymbol{\lambda}}$ as:*

$$\tilde{\boldsymbol{\mu}}_j^* = \begin{cases} \boldsymbol{\mu}_j^* & \text{if } \boldsymbol{\mu}_j^* \neq 0 \\ 1 & \text{otherwise,} \end{cases} \tag{29}$$

and

$$\hat{\mathbf{d}}_{\boldsymbol{\lambda}} = \operatorname{diag}(\rho \tilde{\boldsymbol{\mu}}^*)^{-1} \Big(-\Big(\frac{\partial \ell}{\partial\, \mathbf{z}^*} \Big)^T - \mathbf{Q}\, \hat{\mathbf{d}}_{\mathbf{x}} - \mathbf{A}^T\, \hat{\mathbf{d}}_{\boldsymbol{\eta}} \Big). \tag{30}$$

Then the gradients of the loss function, ℓ, with respect to problem variables $\mathbf{l}$ and $\mathbf{u}$ are given by:

$$\frac{\partial \ell}{\partial\, \mathbf{l}} = \operatorname{diag}(\boldsymbol{\lambda}_-^*) \hat{\mathbf{d}}_{\boldsymbol{\lambda}} \qquad \frac{\partial \ell}{\partial\, \mathbf{u}} = -\operatorname{diag}(\boldsymbol{\lambda}_+^*) \hat{\mathbf{d}}_{\boldsymbol{\lambda}}. \tag{31}$$

We now have a framework for computing the gradient with respect to all problem variables by implicit differentiation of the fixed-point mapping of the transformed ADMM iterations. We re-iterate that the implicit differentiation of the KKT conditions requires solving a system of equations on the order of $3d_z + d_{eq}$. In contrast, the fixed-point iteration, presented in Equation (22) is of dimension $d_z + d_{eq}$. As we will demonstrate shortly, reducing the dimension of the fixed-point mapping results in a considerable improvement in computational efficiency in the backward-pass.

3 Computational experiments

We present several experimental results that highlight the computational efficiency and performance accuracy of the ADMM-layer. In all experiments, computational efficiency is measured by the average runtime (in seconds), required to implement the forward-pass and backward-pass algorithms of each model. We compare across 4 models:

1. **ADMM Unroll:** ADMM in the forward-pass and unrolled differentiation in the backward-pass.
2. **ADMM KKT:** ADMM in the forward-pass and KKT implicit differentiation in the backward-pass.
3. **ADMM FP:** ADMM in the forward-pass and fixed-point implicit differentiation in the backward-pass.
4. **OptNet:** primal-dual interior-point method in the forward-pass and efficient KKT implicit differentiation in the backward-pass.

Both the ADMM and interior-point solvers terminate when the L_2 norms of the primal and dual residuals are sufficiently small (i.e. less than some user-defined tolerance ϵ_{tol}). In many applications, however, it is not always necessary to solve the batch QPs exactly during training. We therefore consider and compare the computational efficiency of each model over several stopping tolerances: $\epsilon_{\text{tol}} \in \{10^{-1}, 10^{-3}, 10^{-5}\}$. Going forward, the model label 'ADMM FP 3', for example, denotes the ADMM FP model with a stopping tolerance of 10^{-3}.

Lastly, first-order methods are known to be vulnerable to ill-conditioned problems and the resulting convergence rates can vary significantly when the data and algorithm parameters (ρ) are poorly scaled. Many first-order solvers therefore implement a preconditioning and problem scaling initialization step (see for example [36, 38, 40]). In our case, however, the QP problem variables are parameterized and are therefore expected to change at each epoch. Scaling and conditioning the problem variables at each epoch would potentially result in excessive computational overhead. As a result, our ADMM-layer implementation does not include a preconditioning step. Instead, in all experiments presented below, we normalize the problem data to have approximately unit standard deviation (on average) and manually scale the problem variables: $\mathbf{p}$, $\mathbf{Q}$, $\mathbf{A}$ and $\mathbf{b}$ where appropriate. We find that for unit-scaled problem data, a value of $\rho \in \{0.10, 1.0\}$ provides a consistent rate of convergence. Indeed, an efficient and dynamic preconditioning and scaling routine is an interesting area of future research.

All experiments are conducted on an Apple Mac Pro computer (2.7 GHz 12-Core Intel Xeon E5,128 GB 1066 MHz DDR3 RAM) running macOS 'Catalina'. All computations are run on an unloaded, single-threaded CPU. The software was written using the R programming language (version 4.0.0) and torch (version 0.6.0).

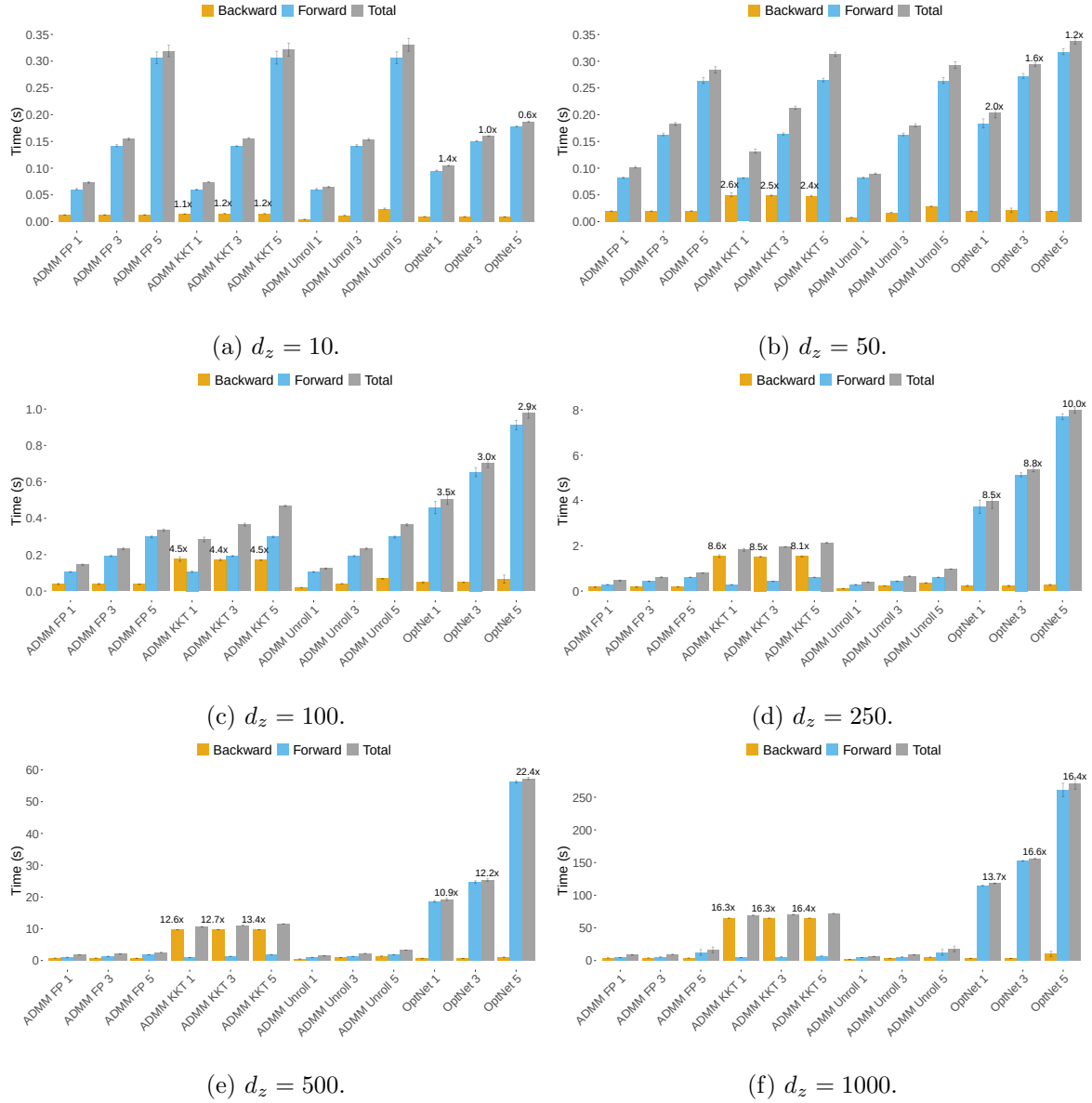
3.1 Experiment 1: ADMM-layer performance

We conduct an experiment comparing the computational efficiency of the ADMM and OptNet models with various stopping tolerances. We randomly generate problem data of dimension:

$$d_z \in \{10, 50, 100, 250, 500, 1000\}.$$

and for each trial implement the forward and backward algorithms on a mini-batch size of 128. Problem variables are generated as follows. We set $\mathbf{Q} = \frac{1}{2d_z}\mathbf{U}^T\mathbf{U}$ where entries of $\mathbf{U} \in \mathbb{R}^{2d_z \times d_z}$ are sampled from a standard normal distribution. We randomly generate $\mathbf{p}$ by sampling from the standard normal distribution and randomly generate $\mathbf{l}$ and $\mathbf{u}$ by randomly sampling from the uniform distribution with domain $[-2, -1]$ and $[1, 2]$, respectively . Finally we set $\mathbf{A} = \mathbf{1}$ and $\mathbf{b} = 1$.

Figure 1 provides the average runtime and 95%-ile confidence interval, evaluated over 10 trials, of the forward and backward-pass algorithms. We make several important observations. First, for small scale problems ($d_z < 100$), there is negligible performance differences across all methods of the same stopping tolerance. As expected, the total runtime increases as the required stopping tolerance decreases. For medium scaled problems ($100 < d_z < 1000$) we observe a substantial performance degradation in both the ADMM-KKT models and the OptNet models, in comparison to the ADMM-FP and ADMM-Unroll models. Specifically, the ADMM-KKT model exhibits an increase in computation time that is anywhere from 4 to 16 times larger than the corresponding ADMM fixed-point backward-pass implementation. This result is not surprising as the ADMM-KKT backward-pass algorithm must first form and then factorize the KKT system of equations, which is of dimension $3d_z + d_{eq}$. In contrast, the fixed-point backward-pass routine solves a system of equations of size $d_z + d_{eq}$ and is shown to be comparable in computational efficiency to the OptNet backward-pass algorithm. Furthermore, for problems of size $d_z \geq 250$, we note that the ADMM-FP and ADMM-Unroll models are approximately an order of magnitude more efficient than the corresponding OptNet models. For example, when $d_z = 1000$ and $\epsilon_{\text{tol}} = 10^{-3}$, the total runtime for the OptNet model is 150 seconds, whereas the total runtime for the ADMM model is less than 10 seconds; over an order of magnitude faster. This increase in computational performance will ultimately enable training architectures that can practically support substantially larger quadratic optimization problems. Lastly, we note that while the ADMM-unroll algorithm is relatively efficient, it requires a significantly larger memory footprint, which may be impractical in some settings. Furthermore, we observe that the as the stopping tolerance decreases the computation time of the unrolled backward-pass increases. In contrast, the fixed-point implicit differentiation method is invariant to the number of 'inner' iterations performed in the ADMM forward-pass. Going forward, we choose to work with the ADMM fixed-point model as it is efficient from both a computational and memory standpoint.



(a) $d_z = 10$. (b) $d_z = 50$.

(c) $d_z = 100$. (d) $d_z = 250$.

(e) $d_z = 500$. (f) $d_z = 1000$.

Figure 1: Computational performance of ADMM-FP, ADMM-KKT, ADMM-Unroll and Optnet for various problem sizes, d_z, and stopping tolerances. Batch size = 128.

3.2 Experiment 2: learning p

We now consider a full training experiment whereby the objective is to learn a parameterized model for the variable $\mathbf{p}$, that is optimal in the context of the remaining QP problem variables. This problem was considered by Donti et al. [19] with applications to power scheduling and battery storage, and more recently by Butler and Kwon [14] for optimal return forecasting within the context of a mean-variance portfolio. We refer to the aforementioned work for more details.

The learning process can be posed as a bi-level optimization program where the objective is to learn a parameter $\boldsymbol{\theta}$ in order to minimize the average QP loss induced by the optimal decision policies $\{\mathbf{z}^*(\boldsymbol{\theta})^{(i)}\}_{i=1}^m$. Program (32) is referred to as an integrated predict and optimize (IPO) model as the prediction model for $\mathbf{p}$ is fully integrated with the resulting down-stream decision-based optimization model.

$$\begin{aligned}
\underset{\boldsymbol{\theta}}{\text{minimize}} \quad & \frac{1}{m}\sum_{i=1}^{m}\Big(\mathbf{z}^*(\boldsymbol{\theta})^{T^{(i)}}\,\mathbf{p}^{(i)} + \frac{1}{2}\,\mathbf{z}^*(\boldsymbol{\theta})^{T^{(i)}}\,\mathbf{Q}^{(i)}\,\mathbf{z}^*(\boldsymbol{\theta})^{(i)}\Big) \\
\text{subject to} \quad & \mathbf{z}^*(\boldsymbol{\theta})^{(i)} = \underset{\mathbf{z}}{\text{argmin}} - \mathbf{z}^T\,\hat{\mathbf{p}}(\boldsymbol{\theta})^{(i)} + \frac{1}{2}\,\mathbf{z}^T\,\hat{\mathbf{Q}}^{(i)}\,\mathbf{z} \quad \forall i = 1, ..., m \\
& \mathbf{A}\,\mathbf{z}^*(\boldsymbol{\theta})^{(i)} = \mathbf{b} \quad \forall i = 1, ..., m \\
& \mathbf{l} \leq \mathbf{z}^*(\boldsymbol{\theta})^{(i)} \leq \mathbf{u} \quad \forall i = 1, ..., m.
\end{aligned} \tag{32}$$

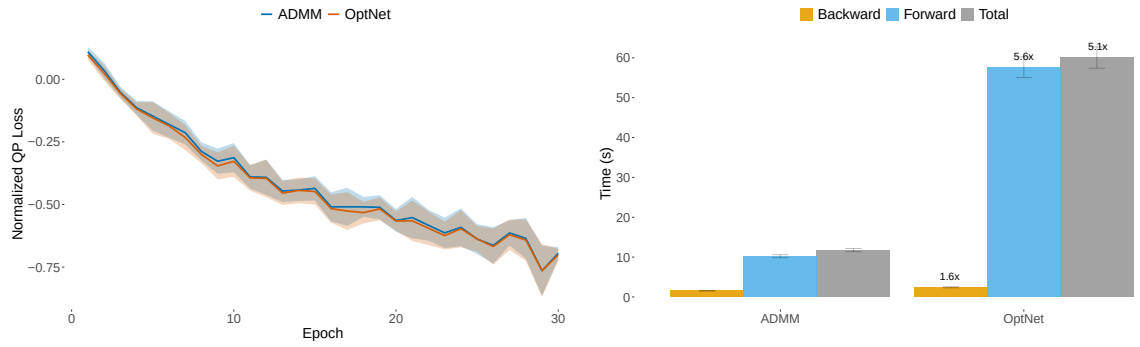
Here $\mathbf{p}^{(i)}$ and $\mathbf{Q}^{(i)}$ denote the ground truth problem data and are generated as follows. We let $\mathbf{p}^{(i)} \sim \mathcal{N}(\mathbf{w}^{T^{(i)}}\,\boldsymbol{\theta}_0 + \tau\,\boldsymbol{\epsilon}^{(i)}, \mathbf{Q})$ where $\mathbf{Q} \in \mathbb{R}^{d_z \times d_z}$ has entry (j,k) equal to $\rho_{\mathbf{p}}^{|j-k|}$. We set $\rho_{\mathbf{p}} = 0.50$ and generate the auxiliary feature data from the standard normal distribution, $\mathbf{w}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{\mathbf{w}})$. The residuals, $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, preserve the desired correlation structure and the scalar value τ controls the signal-to-noise level. All experiments target a signal-to-noise level of 0.10.

We let $\hat{\mathbf{p}}(\boldsymbol{\theta})^{(i)}$ denote the estimate of $\mathbf{p}^{(i)}$ according to the linear model:

$$\hat{\mathbf{p}}(\boldsymbol{\theta})^{(i)} = \mathbf{w}^{T^{(i)}}\,\boldsymbol{\theta}\,. \tag{33}$$

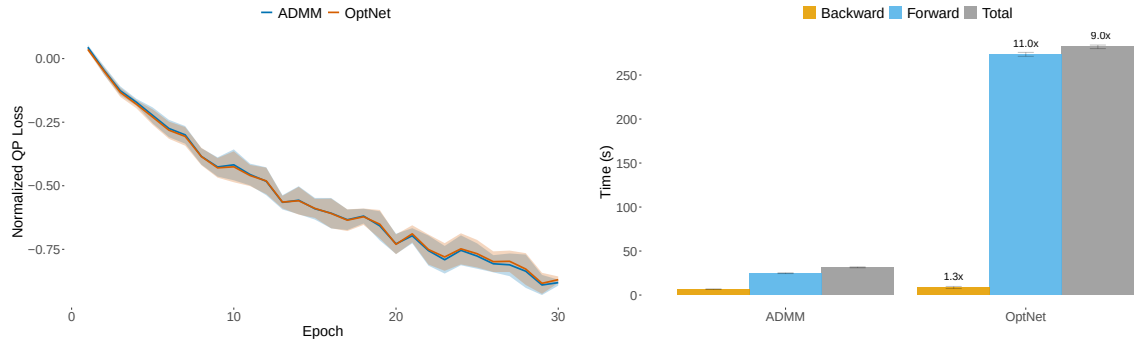
The bound constraints, $\mathbf{l}$ and $\mathbf{u}$, are generated by randomly sampling from the uniform distribution with domain $[-1, 0]$ and $[0, 1]$, respectively and we set $\mathbf{A} = \mathbf{1}$ and $\mathbf{b} = 1$. In all experiments we set the stopping tolerance to $\epsilon_{\text{tol}} = 10^{-3}$.

We randomly generate problem data of dimension $d_z \in \{250, 500, 1000\}$. The training process for each trial consists of 30 epochs with a mini-batch size of 32. Figures 2(a) - 4(a) report the average training loss at each epoch and the 95%-ile confidence interval, evaluated over 10 independent trials. Observe that the loss curves for the ADMM model and OptNet model are almost identical at each epoch, suggesting that the training accuracy of the ADMM model and OptNet model are equivalent. Conversely, Figures 2(b) - 4(b), compare the average and 95%-ile confidence interval time spent executing the forward and backward pass algorithms. When $d_z = 250$ the ADMM model is shown to be approximately 5 times faster than the OptNet model. Furthermore, when $d_z = 500$ and $d_z = 1000$, the ADMM model is a full order of magnitude faster than the OptNet model. More concretely, when $d_z = 1000$ the entire learning process takes approximately 1600 seconds to train the OptNet model, but less than 130 seconds to train the ADMM model to an equal level of accuracy.



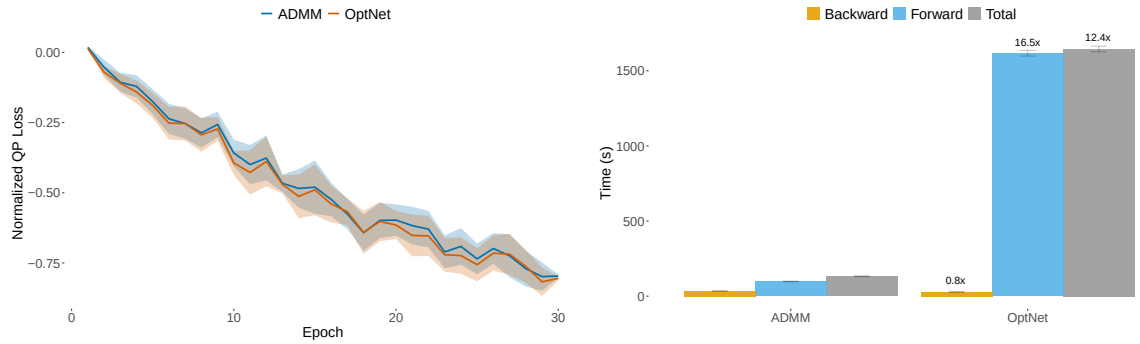
(a) Training Loss. (b) Computational Performance.

Figure 2: Training loss and computational performance for learning $\mathbf{p}$. Batch size $= 32$ and $d_z = 250$.



(a) Training Loss (b) Computational Performance

Figure 3: Training loss and computational performance for learning $\mathbf{p}$. Batch size $= 32$ and $d_z = 500$.



(a) Training Loss (b) Computational Performance

Figure 4: Training loss and computational performance for learning $\mathbf{p}$. Batch size $= 32$ and $d_z = 1000$.

3.3 Experiment 3: learning A

We now present a real-world experiment from portfolio optimization whereby the objective is to learn a parameterized model for the variable $\mathbf{A}$. We consider an asset universe of $d_z = 255$ liquid US stocks traded on major U.S. exchanges (NYSE, NASDAQ, AMEX, ARCA). The universe is summarized in Table 3, with representative stocks from each of the Global Industry Classification Standard (GICS) sectors. Weekly price data is given from January 1990 through December 2020, and is provided by Quandl.

We denote the matrix of weekly return observations as $\mathbf{A} = [\mathbf{a}^{(1)}, \mathbf{a}^{(2)}, ..., \mathbf{a}^{(m)}] \in \mathbb{R}^{m \times d_z}$ with $m > d_z$. Let $\mathbf{Q}^{(i)} \in \mathbb{R}^{d_z \times d_z}$ denote the symmetric positive definite covariance matrix of asset returns. We define the portfolio $\mathbf{z}^{(i)} \in \mathbb{R}^{d_z}$, where the element, ${\mathbf{z}^{(i)}}_j$, denotes the proportion of total capital invested in the jth asset at time i.

We define the Sharpe ratio at observation i as the ratio of portfolio return to portfolio risk, where risk is measured by the portfolio volatility (standard deviation).

$$\mathrm{S_R}^{(i)} = \frac{\mathbf{a}^{T^{(i)}}\, \mathbf{z}^{(i)}}{\sqrt{\mathbf{z}^{T^{(i)}}\, \mathbf{Q}^{(i)}\, \mathbf{z}^{(i)}}} \tag{34}$$

We consider a long-only ($\mathbf{z}^{(i)} \geq 0$), fully invested ($\mathbf{1}^T\, \mathbf{z}^{(i)} = 1$) max-Sharpe portfolio optimization, presented in Program (35):

$$\begin{aligned} \underset{\mathbf{z}}{\text{maximize}} \quad & \frac{\mathbf{a}^{T^{(i)}}\, \mathbf{z}}{\sqrt{\mathbf{z}^T\, \mathbf{Q}^{(i)}\, \mathbf{z}}} \\ \text{subject to} \quad & \mathbf{1}^T\, \mathbf{z} = 1, \quad 0 \leq \mathbf{z} \leq 1 \end{aligned} \tag{35}$$

Observe, however, that the Sharpe ratio is not convex in $\mathbf{z}$ but is homogeneous of degree zero. We follow Cornuejols and Tutuncu [15] and re-cast Program (35) as a convex quadratic optimization program:

$$\begin{aligned} \underset{\mathbf{z}}{\text{minimize}}\, \frac{1}{2} \quad & \mathbf{z}^T\, \mathbf{Q}^{(i)}\, \mathbf{z} \\ \text{subject to} \quad & \mathbf{a}^{T^{(i)}}\, \mathbf{z} = 1, \quad \mathbf{z} \geq 0. \end{aligned} \tag{36}$$

Note that the fully-invested constraint can be enforced by normalizing the optimal weights, $\mathbf{z}^*$.

As before, the learning process can be posed as a bi-level optimization program where the objective is to learn a parameter $\boldsymbol{\theta}$ and the associated constraints, $\hat{\mathbf{a}}(\boldsymbol{\theta})^{(i)}$, in order to maximize the average realized Sharpe ratio induced by the optimal decision policies $\{\mathbf{z}^*(\boldsymbol{\theta})^{(i)}\}_{i=1}^m$.

$$\begin{aligned} \underset{\boldsymbol{\theta}}{\text{minimize}} \quad & -\frac{1}{m} \sum_{i=1}^{m} \frac{\mathbf{a}^{T^{(i)}}\, \mathbf{z}^*(\boldsymbol{\theta})}{\sqrt{\mathbf{z}^*(\boldsymbol{\theta})^{T^{(i)}}\, \mathbf{Q}^{(i)}\, \mathbf{z}^*(\boldsymbol{\theta})^{(i)}}} \\ \text{subject to} \quad & \mathbf{z}^*(\boldsymbol{\theta}) = \underset{\mathbf{z}}{\operatorname{argmin}}\, \frac{1}{2}\, \mathbf{z}^T\, \mathbf{Q}^{(i)}\, \mathbf{z} \quad \forall i = 1, ..., m \\ & \hat{\mathbf{a}}(\boldsymbol{\theta})^{T^{(i)}}\, \mathbf{z}^*(\boldsymbol{\theta})^{(i)} = 1, \quad \mathbf{1}^T\, \mathbf{z}^*(\boldsymbol{\theta})^{(i)} = 1, \quad \mathbf{z}^*(\boldsymbol{\theta})^{(i)} \geq 0 \quad \forall i = 1, ..., m \end{aligned} \tag{37}$$

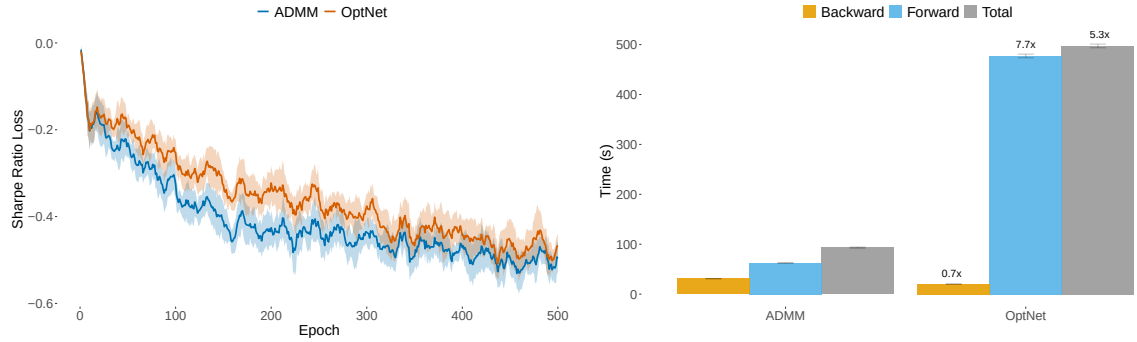
In reality, we do not know the true value of $\mathbf{a}^{(i)}$ at decision time and instead we estimate $\mathbf{a}^{(i)}$ through associated auxiliary feature variables $\mathbf{w}^{(i)} \in \mathbb{R}^{d_w}$. Again we consider a linear model of the form:

$$\hat{\mathbf{a}}^{(i)} = \boldsymbol{\theta}^T \mathbf{w}^{(i)} \,. \tag{38}$$

In this experiment, asset returns, $\mathbf{a}^{(i)}$ are modelled using the well-known Famma-French Five (FF5) factor model [22], provided by the Kenneth R. French data library.

The goal is to observe the training and out-of-sample performance of the ADMM model in comparison to the OptNet model. As a benchmark, we include the out-of-sample performance of an equally weighted portfolio, and a max-Sharpe portfolio where $\boldsymbol{\theta}$ is fit by ordinary least-squares (OLS). All experiments are trained on data from January 1990 through December 2014. The out-of-sample period begins in January 2015 and ends in December 2020. Portfolios are formed at the close of each week, and rebalanced on a weekly basis.

The training process for each trial consists of 500 epochs with a mini-batch size of 32. Portfolio models are fit to an accuracy of $\epsilon_{\text{tol}} = 10^{-4}$ in training, and a higher accuracy of $\epsilon_{\text{tol}} = 10^{-6}$ in the out-of-sample period in order to guarantee strict adherence to the constraint set. Figure 5(a) reports the average training loss at each epoch and the 95%-ile confidence interval, evaluated over 10 independent trials. Once again, we observe that the loss curves for the ADMM model and OptNet model are very similar. Interestingly, we observe that the ADMM model produces a consistently lower average training loss. Recall that both models use implicit differentiation to compute the relevant gradient, which assumes an exact fixed point at each optimal solution $\mathbf{z}^*(\boldsymbol{\theta})^{(i)}$. In practice, each $\mathbf{z}^*(\boldsymbol{\theta})^{(i)}$ is only approximately optimal, to within a tolerance ϵ_{tol}, and therefore differentiating at a solution that is not an exact fixed point will result in small errors in the gradient that likely explain the observed difference. That said, the training loss profile of the ADMM and OptNet models are very similar, and the final models achieve approximately equal loss after 500 epochs. Figure 5(b) compares the average and 95%-ile confidence interval of the total time spent executing the forward and backward pass algorithms during training. Once again we observe that the ADMM model is shown to be approximately 5 times faster than the OptNet model and requires less than 100 seconds to train.



(a) Training Loss

(b) Computational Performance

Figure 5: Training loss and computational performance for learning $\mathbf{A}$ on US stock data. Batch size $= 32$ and $d_z = 255$.

Figure 6 reports the out-of-sample equity growth of the ADMM IPO max-Sharpe portfolio, Equal Weight portfolio, OLS max-Sharpe portfolio and OptNet IPO max-Sharpe portfolio. The out-of-sample economic performance metrics are reported in Table 1. First, observe that all max-Sharpe models outperform the Equal Weight benchmark on an absolute and risk-adjusted basis. Furthermore, the ADMM and OptNet IPO max-Sharpe models achieve an out-of-sample Sharpe ratio that is approximately 50% higher than that of the naive 'predict, then optimize' OLS max-Sharpe model, thus highlighting the benefit of training a fully integrated system. Lastly, the ADMM model achieves a marginally higher out-of-sample Sharpe ratio in comparison to the OptNet model, though the difference is not statistically significant.

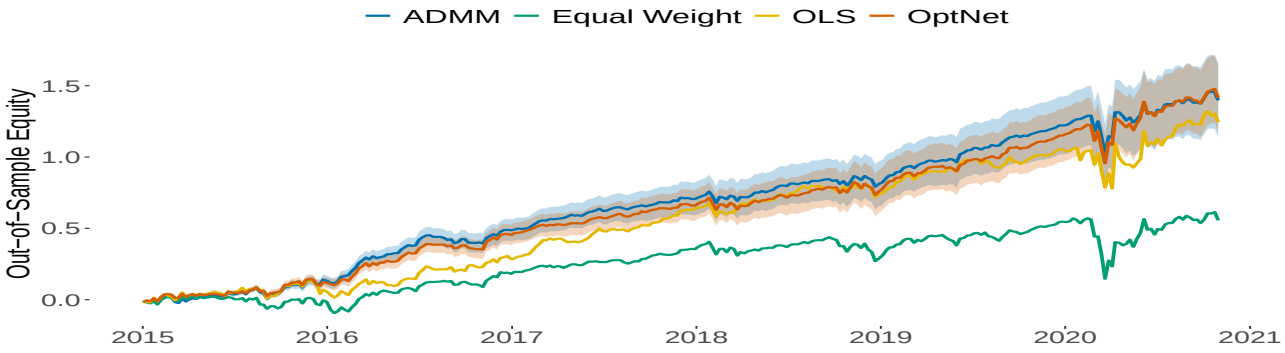


Figure 6: Out-of-sample equity growth for ADMM IPO max-Sharpe portfolio, Equal Weight portfolio, OLS max-Sharpe portfolio and OptNet IPO max-Sharpe portfolio.

	ADMM	Equal Weight	OLS	OptNet
Mean	0.2382	0.0950	0.2122	0.2413
Volatility	0.1777	0.1950	0.2435	0.1880
Sharpe Ratio	1.3407	0.4872	0.8747	1.2836

Table 1: Out-of-sample economic performance metrics for ADMM IPO max-Sharpe portfolio, Equal Weight portfolio, OLS max-Sharpe portfolio and OptNet IPO max-Sharpe portfolio.

3.4 Experiment 4: learning Q

We consider another real-world experiment from portfolio optimization whereby the objective is to learn a parameterized model for the variable $\mathbf{Q}$. We use the same asset universe of $d_z = 255$ liquid US stocks from Experiment 3 and an identical experimental design. Here, we consider the long-only, fully-invested minimum variance portfolio optimization, described in Program (39).

$$\begin{array}{ll} \underset{\mathbf{z}}{\text{minimize}} \dfrac{1}{2} & \mathbf{z}^T \mathbf{Q}^{(i)} \mathbf{z} \\ \text{subject to} & \mathbf{1}^T \mathbf{z} = 1, \quad 0 \leq \mathbf{z} \leq 1. \end{array} \tag{39}$$

As before, the learning process is posed as a bi-level optimization program where the objective is to learn a parameter $\boldsymbol{\theta}$ and the associated covariance matrix, $\hat{\mathbf{Q}}(\boldsymbol{\theta})^{(i)}$, in order to minimize the average realized variance induced by the optimal decision policies $\{\mathbf{z}^*(\boldsymbol{\theta})^{(i)}\}_{i=1}^m$.

$$\begin{aligned} \underset{\boldsymbol{\theta}}{\text{minimize}} \quad & \frac{1}{m}\sum_{i=1}^{m} \mathbf{z}^*(\boldsymbol{\theta})^{T^{(i)}}\, \mathbf{Q}^{(i)}\, \mathbf{z}^*(\boldsymbol{\theta})^{(i)} \\ \text{subject to} \quad & \mathbf{z}^*(\boldsymbol{\theta}) = \underset{\mathbf{z}}{\text{argmin}}\, \frac{1}{2}\, \mathbf{z}^T\, \hat{\mathbf{Q}}(\boldsymbol{\theta})^{(i)}\, \mathbf{z} \quad \forall i = 1, ..., m \\ & \mathbf{1}^T\, \mathbf{z}^*(\boldsymbol{\theta})^{(i)} = 1, \quad 0 \leq \mathbf{z}^*(\boldsymbol{\theta})^{(i)} \leq 1 \quad \forall i = 1, ..., m \end{aligned} \tag{40}$$

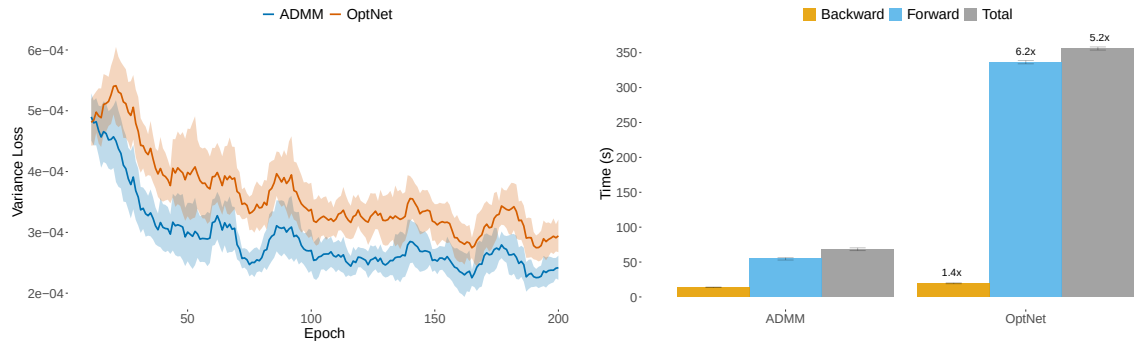
Asset returns, $\mathbf{a}^{(i)}$ are modelled using the Famma-French Five (FF5) factor model. We follow Butler and Kwon [13] and model $\mathbf{w}^{(i)}$ according to a multivariate GARCH$(1,1)$ process with constant correlation. We let $\hat{\mathbf{W}}^{(i)}$ denote the the time-varying covariance estimate of the auxiliary feature variables. We therefore model the stock covariance matrix as follows:

$$\hat{\mathbf{a}}^{(i)} = \boldsymbol{\theta}^T\, \mathbf{w}^{(i)}, \quad \hat{\mathbf{Q}}^{(i)} = \boldsymbol{\theta}^T\, \hat{\mathbf{W}}^{(i)}\, \boldsymbol{\theta} + \hat{\mathbf{F}}, \tag{41}$$

where $\hat{\mathbf{F}}$ denotes the diagonal matrix of residual variances.

Again, the goal is to observe the training and out-of-sample performance of the ADMM model in comparison to the OptNet model. As a benchmark, we include the out-of-sample performance of the equal weight portfolio, and a minimum variance portfolio where $\boldsymbol{\theta}$ is fit by OLS. The training process for each trial consists of 200 epochs with a mini-batch size of 32. Portfolio models are fit to an accuracy of $\epsilon_{\text{tol}} = 10^{-4}$ in training, and a higher accuracy of $\epsilon_{\text{tol}} = 10^{-6}$ in the out-of-sample period.

Figure 7(a) reports the average training loss at each epoch and the 95%-ile confidence interval, evaluated over 10 independent trials. We observe that the loss curves for the ADMM model and OptNet model are very similar, thus highlighting the accuracy of the ADMM-layer. Once again we observe that the ADMM model produces a consistently lower average training loss and we refer to the discussion in Experiment 3 for a likely explanation. Figure 7(b) compares the average and 95%-ile confidence interval of the total time spent executing the forward and backward pass algorithms in training. As before, we observe that the ADMM model requires approximately 60 seconds to train and is approximately 5 times faster than the OptNet model, which requires over 300 seconds.



(a) Training Loss

(b) Computational Performance

Figure 7: Training loss and computational performance for learning $\mathbf{Q}$ on US stock data. Batch size $= 32$ and $d_z = 255$.

Finally, Figure 8 reports the out-of-sample equity growth of the ADMM IPO minimum variance portfolio, Equal Weight portfolio, OLS minimum variance portfolio and OptNet IPO minimum variance portfolio. The out-of-sample economic performance metrics are reported in Table 2. Again, observe that all minimum-variance models outperform the Equal Weight benchmark on an absolute and risk-adjusted basis. Furthermore, the ADMM and OptNet IPO minimum variance models achieve an out-of-sample volatility that is approximately 25% lower and Sharpe ratio that is approximately 35% higher than that of the naive 'predict, then optimize' OLS minimum variance model. These results are broadly consistent with the findings in Butler and Kwon [13], who consider an identical stock universe but with considerably smaller portfolios ($d_z \leq 100$). Our ADMM model, on the other hand, is able to overcome the computational challenges in the medium to large scale limit (described in Butler and Kwon [13]), without any apparent loss in performance accuracy.

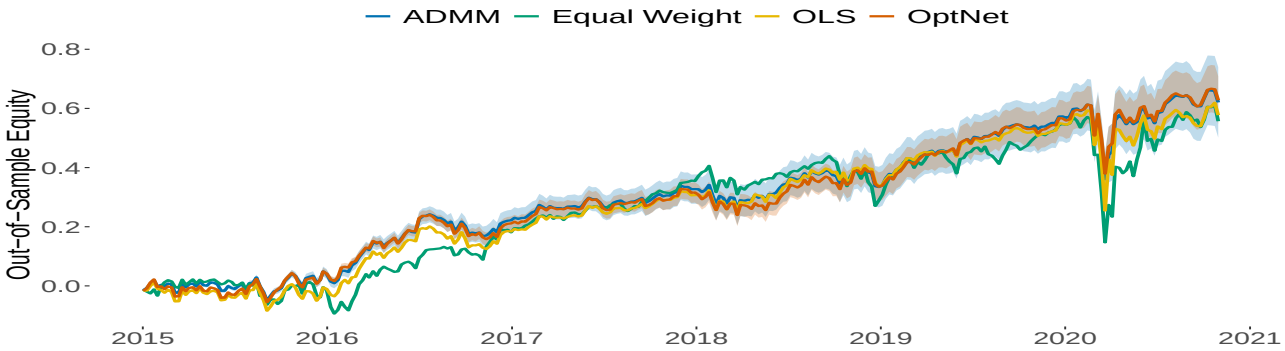


Figure 8: Out-of-sample equity growth for ADMM IPO minimum variance portfolio, Equal Weight portfolio, OLS minimum variance portfolio and OptNet IPO minimum variance portfolio.

	ADMM	Equal Weight	OLS	OptNet
Mean	0.1058	0.0950	0.0984	0.1070
Volatility	0.1420	0.1950	0.1786	0.1443
Sharpe Ratio	0.7446	0.4872	0.5510	0.7418

Table 2: Out-of-sample economic performance metrics for ADMM IPO minimum variance portfolio, Equal Weight portfolio, OLS minimum variance portfolio and OptNet IPO minimum variance portfolio.

4 Conclusion and future work

In this paper, we provide a novel and efficient framework for differentiable constrained quadratic programming. Our differentiable quadratic programming layer is built on top of the ADMM algorithm, which for medium to large scale problems is shown to be approximately an order of magnitude more efficient than the interior point implementation of the OptNet layer. The backward-pass algorithm computes the relevant problem variable gradients by implicit differentiation of a modified fixed-point iteration, which is computationally favorable to KKT implicit differentiation and memory efficient in comparison to standard unrolled differentiation. Numerical results, using both simulated and real problem data, demonstrates the efficacy of the ADMM layer, which for medium to large scale problems exhibits state-of-the art accuracy and improved computational performance.

Our experimental results should be interpreted as a proof-of-concept and we acknowledge that further testing on alternative data sets or with different problem variable assumptions is required in order to better determine the efficacy of the ADMM layer as a general purpose solver. Indeed, there is a plethora of areas for active research and algorithm improvement. First, our ADMM layer currently only supports linear equality and box inequality constraints, whereas the OptNet layer supports general linear inequality constraints. Indeed, incorporating more general inequality constraints as well as augmenting the QP with parameterized regularization norms is an active area of research. Secondly, as discussed earlier, the ADMM algorithm is known to be vulnerable to ill-conditioned problems and the resulting convergence rates can vary significantly when the data and algorithm parameter, ρ, are poorly scaled. To overcome this, most first-order solvers implement a preconditioning and scaling initialization step. Currently, our ADMM-layer implementation does not support preconditioning and scaling, which is challenged by virtue of the fact that the problem data is expected to change at each epoch of training. Instead, we leave it to the user to select ρ and manually scale the problem data and acknowledge that preconditioning, scaling and automatic parameter selection is an important area of future development.

Furthermore, there are several heuristic methods that could be implemented in order to improve the convergence rates and computational efficiency of our ADMM forward-pass. For example, acceleration methods, such as Andersen acceleration [5], have recently been shown to improve the convergence rates of first-order solvers [39, 44]. Indeed, applying acceleration methods to the modified fixed-point algorithm presented in this paper provides an numerically efficient scheme for potentially improving the convergence rate of the ADMM algorithm and is an interesting area of future research. Alternatively, methods that hybridize the efficiency of first-order methods with the precision of interior-point methods, such solution polishing and refinement [12], is another area of future exploration. Nonetheless,

our proposed ADMM layer is shown to be highly effective and in its current form can be instrumental for efficiently solving real-world medium and large scale learning problems.

References

[1] Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J. Zico Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems*, volume 32, pages 9562–9574. Curran Associates, Inc., 2019.

[2] Akshay Agrawal, Shane Barratt, Stephen Boyd, Enzo Busseti, and Walaa M. Moursi. Differentiating through a cone program, 2020. URL `http://arxiv.org/abs/1703.00443`.

[3] Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. *CoRR*, abs/1703.00443, 2017. URL `http://arxiv.org/abs/1703.00443`.

[4] Brandon Amos, Ivan Dario Jimenez Rodriguez, Jacob Sacks, Byron Boots, and J. Zico Kolter. Differentiable mpc for end-to-end planning and control, 2019.

[5] Donald G. M. Anderson. Iterative procedures for nonlinear integral equations. *J. ACM*, 12: 547–560, 1965.

[6] David Belanger, Bishan Yang, and Andrew McCallum. End-to-end learning for structured prediction energy networks, 2017.

[7] Dimitris Bertsimas and Nathan Kallus. From predictive to prescriptive analytics. *Management Science*, 66(3):1025–1044, 2020.

[8] F. Black and R. Litterman. Asset allocation combining investor views with market equilibrium. *Journal of Fixed Income*, 1(2):7–18, 1991.

[9] Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-Lopez, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and modular implicit differentiation, 2021.

[10] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[11] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3:1–122, 01 2011. doi: 10.1561/2200000016.

[12] E. Busseti, W. Moursi, and S. Boyd. Solution refinement at regular points of conic problems, 2018.

[13] Andrew Butler and Roy Kwon. Covariance estimation for risk-based portfolio optimization: an integrated approach. *Social Science Research Network*, 1(1), 2021.

[14] Andrew Butler and Roy H. Kwon. Integrating prediction in mean-variance portfolio optimization. *arXiv 2102.09287*, pages 1–33, 2021.

[15] Gerard Cornuejols and Reha Tutuncu. *Optimization Methods in Finance*. Cambridge University Press, 2006.

[16] Steven Diamond, Vincent Sitzmann, Felix Heide, and Gordon Wetzstein. Unrolled optimization with deep priors, 2018.

[17] Justin Domke. Generic methods for optimization-based modeling. In *AISTATS*, 2012.

[18] Asen Dontchev and R Rockafellar. *Implicit Functions and Solution Mappings: A View from Variational Analysis*. 01 2009. ISBN 978-0-387-87820-1. doi: 10.1007/978-0-387-87821-8.

[19] Priya L. Donti, Brandon Amos, and J. Zico Kolter. Task-based End-to-end Model Learning. *CoRR*, abs/1703.04529, 2017. URL `http://arxiv.org/abs/1703.04529`.

[20] Adam N. Elmachtoub and Paul Grigas. Smart predict, then optimize. *arXiv*, 2020.

[21] Adam N. Elmachtoub, Jason Cheuk Nam Liang, and Ryan McNellis. Decision trees for decision-making under the predict-then-optimize framework, 2020.

[22] Eugene F. Fama and Kenneth R. French. A five-factor asset pricing model. *Journal of Financial Economics*, 116(1):1 – 22, 2015. ISSN 0304-405X. doi: https://doi.org/10.1016/j.jfineco.2014.10.010. URL `http://www.sciencedirect.com/science/article/pii/S0304405X14002323`.

[23] Jean Feng and Noah Simon. Gradient-based regularization parameter selection for problems with non-smooth penalty functions, 2017.

[24] Daniel Gabay and Bertrand Mercier. A dual algorithm for the solution of nonlinear variational problems via finite element approximation. *Computers & Mathematics With Applications*, 2: 17–40, 1976.

[25] R. Ganti and Alexander G. Gray. Cake: Convex adaptive kernel density estimation. In *AISTATS*, 2011.

[26] Roland Glowinski and A. Marroco. Sur l'approximation, par éléments finis d'ordre un, et la résolution, par pénalisation-dualité d'une classe de problèmes de dirichlet non linéaires. 1975.

[27] D. Goldfarb and Shucheng Liu. An o(n3l) primal interior point algorithm for convex quadratic programming. *Mathematical Programming*, 49:325–340, 1991.

[28] Paul Grigas, Meng Qi, Zuo-Jun, and Shen. Integrated conditional estimation-optimization, 2021.

[29] Michael Ho, Zheng Sun, and Jack Xin. Weighted elastic net penalized mean-variance portfolio design and computation. *SIAM Journal on Financial Mathematics*, 6(1):1220–1244, 2015.

[30] Seung-Jean Kim, K. Koh, M. Lustig, Stephen Boyd, and Dimitry Gorinevsky. An interior-point method for large-scale l1-regularized least squares. *Selected Topics in Signal Processing, IEEE Journal of*, 1:606 – 617, 01 2008. doi: 10.1109/JSTSP.2007.910971.

[31] Jonathan Lorraine and David Duvenaud. Stochastic hyperparameter optimization through hypernetworks, 2018.

[32] Jayanta Mandi and Tias Guns. Interior point solving for lp-based prediction+optimisation, 2020.

[33] Jaynta Mandi, Emir Demirovic, Peter. J Stuckey, and Tias Guns. Smart predict-and-optimize for hard combinatorial optimization problems, 2019.

[34] H. Markowitz. Portfolio selection. *Journal of Finance*, 7(1):77–91, 1952.

[35] Richard Michaud and Robert Michaud. Estimation error and portfolio optimization: A resampling solution. *Journal of Investment Management*, 6(1):8–28, 2008.

[36] Brendan O'Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding, 2016.

[37] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[38] Michel Schubiger, Goran Banjac, and John Lygeros. Gpu acceleration of admm for large-scale quadratic programming. *Journal of Parallel and Distributed Computing*, 144:55–67, 2020. ISSN 0743-7315. doi: https://doi.org/10.1016/j.jpdc.2020.05.021. URL `https://www.sciencedirect.com/science/article/pii/S0743731520303063`.

[39] Pantelis Sopasakis, Krina Menounou, and Panagiotis Patrinos. Superscs: fast and accurate large-scale conic optimization. pages 1500–1505, 06 2019. doi: 10.23919/ECC.2019.8796286.

[40] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. Osqp: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637?672, Feb 2020. ISSN 1867-2957. doi: 10.1007/s12532-020-00179-2. URL `http://dx.doi.org/10.1007/s12532-020-00179-2`.

[41] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society*, 58(1), 1996.

[42] Andrey Tikhonov. Solution of incorrectly formulated problems and the regularization method. *Soviet Mathematics*, (4):1035–1038, 1963.

[43] Ayse Sinem Uysal, Xiaoyue Li, and John M. Mulvey. End-to-end risk budgeting portfolio optimization with neural networks, 2021.

[44] Homer Walker and Peng Ni. Anderson acceleration for fixed-point iterations. *SIAM J. Numerical Analysis*, 49:1715–1735, 08 2011. doi: 10.2307/23074353.

[45] Xingyu Xie, Jianlong Wu, Zhisheng Zhong, Guangcan Liu, and Zhouchen Lin. Differentiable linearized admm, 2019.

[46] Yan Yang, Jian Sun, Huibin Li, and Zongben Xu. Admm-net: A deep learning approach for compressive sensing mri, 2017.

A Proof of Proposition 1

We define $\mathbf{v}^k = \mathbf{x}^{k+1} + \boldsymbol{\mu}^k$. We can therefore express Equation (14b) as:

$$\mathbf{z}^{k+1} = \Pi(\mathbf{x}^{k+1} + \boldsymbol{\mu}^k) = \Pi(\mathbf{v}^k), \tag{42}$$

and Equation (14c) as:

$$\boldsymbol{\mu}^{k+1} = \boldsymbol{\mu}^k + \mathbf{x}^{k+1} - \mathbf{z}^{k+1} = \mathbf{v}^k - \Pi(\mathbf{v}^k). \tag{43}$$

Substituting Equations (42) and (43) into Equation (14b) gives the desired fixed-point iteration:

$$\begin{bmatrix} \mathbf{v}^{k+1} \\ \boldsymbol{\eta}^{k+2} \end{bmatrix} = \begin{bmatrix} \mathbf{x}^{k+2} + \boldsymbol{\mu}^{k+1} \\ \boldsymbol{\eta}^{k+2} \end{bmatrix} \tag{44}$$

$$= - \begin{bmatrix} \mathbf{Q} + \rho \mathbf{I_v} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{p} - \rho(\mathbf{z}^{k+1} - \boldsymbol{\mu}^{k+1}) \\ -\mathbf{b} \end{bmatrix} + \begin{bmatrix} \boldsymbol{\mu}^{k+1} \\ 0 \end{bmatrix} \tag{45}$$

$$= - \begin{bmatrix} \mathbf{Q} + \rho \mathbf{I_v} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{p} - \rho(2\Pi(\mathbf{v}^k) - \mathbf{v}^k) \\ -\mathbf{b} \end{bmatrix} + \begin{bmatrix} \mathbf{v}^k \\ \boldsymbol{\eta}^{k+1} \end{bmatrix} - \begin{bmatrix} \Pi(\mathbf{v}^k) \\ \boldsymbol{\eta}^{k+1} \end{bmatrix}. \tag{46}$$

B Proof of Proposition 2

We define $F \colon \mathbb{R}^{d_v} \times \mathbb{R}^{d_\eta} \to \mathbb{R}^{d_v} \times \mathbb{R}^{d_\eta}$ as:

$$F(\mathbf{v}, \boldsymbol{\eta}) = - \begin{bmatrix} \mathbf{Q} + \rho \mathbf{I_v} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{p} - \rho(2\Pi(\mathbf{v}) - \mathbf{v}) \\ -\mathbf{b} \end{bmatrix} + \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\eta} \end{bmatrix} - \begin{bmatrix} \Pi(\mathbf{v}) \\ \boldsymbol{\eta} \end{bmatrix}, \tag{47}$$

and let

$$\mathbf{M} = \begin{bmatrix} \mathbf{Q} + \rho \mathbf{I_v} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}. \tag{48}$$

Therefore we have

$$\mathbf{M}\, F(\mathbf{v}, \boldsymbol{\eta}) = - \begin{bmatrix} \mathbf{p} - \rho(2\Pi(\mathbf{v}) - \mathbf{v}) \\ -\mathbf{b} \end{bmatrix} + \mathbf{M} \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\eta} \end{bmatrix} - \mathbf{M} \begin{bmatrix} \Pi(\mathbf{v}) \\ \boldsymbol{\eta} \end{bmatrix}. \tag{49}$$

Taking the partial differentials of Equation (50) with respect to the relevant problem variables therefore gives:

$$\begin{aligned} \mathbf{M}\, \partial F(\mathbf{v}, \boldsymbol{\eta}) &= - \begin{bmatrix} \partial \mathbf{p} \\ -\partial \mathbf{b} \end{bmatrix} + \partial \mathbf{M} \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\eta} \end{bmatrix} - \partial \mathbf{M} \begin{bmatrix} \Pi(\mathbf{v}) \\ \boldsymbol{\eta} \end{bmatrix} - \partial \mathbf{M}\, F(\mathbf{v}, \boldsymbol{\eta}) \\ &= - \begin{bmatrix} \partial \mathbf{p} \\ -\partial \mathbf{b} \end{bmatrix} - \partial \mathbf{M} \left[- \mathbf{M}^{-1} \begin{bmatrix} \mathbf{p} - \rho(2\Pi(\mathbf{v}) - \mathbf{v}) \\ -\mathbf{b} \end{bmatrix} \right] \\ &= - \begin{bmatrix} \partial \mathbf{p} \\ -\partial \mathbf{b} \end{bmatrix} - \partial \mathbf{M} \begin{bmatrix} \mathbf{x}^* \\ \boldsymbol{\eta}^* \end{bmatrix} \\ &= - \begin{bmatrix} \partial \mathbf{p} + \frac{1}{2}(\partial \mathbf{Q}\, \mathbf{x}^* + \partial \mathbf{Q}^T \mathbf{x}^*) + \partial \mathbf{A}^T \boldsymbol{\eta}^* \\ -\partial \mathbf{b} + \partial \mathbf{A}\, \mathbf{x}^* \end{bmatrix}. \end{aligned} \tag{50}$$

From Equation (50) we have that the differential $\partial F(\mathbf{v}, \boldsymbol{\eta})$ is given by:

$$\partial F(\mathbf{v}, \boldsymbol{\eta}) = -\mathbf{M}^{-1} \begin{bmatrix} \partial \mathbf{p} + \frac{1}{2}(\partial \mathbf{Q}\, \mathbf{x}^* + \partial \mathbf{Q}^T \mathbf{x}^*) + \partial \mathbf{A}^T \boldsymbol{\eta}^* \\ -\partial \mathbf{b} + \partial \mathbf{A}\, \mathbf{x}^* \end{bmatrix}. \tag{51}$$

Substituting the gradient action of Equation (51) into Equation (26) and taking the left matrix-vector product of the transposed Jacobian with the previous backward-pass gradient, $\frac{\partial \ell}{\partial \mathbf{z}^*}$, gives the desired result.

$$\begin{bmatrix} \hat{\mathbf{d}}_{\mathbf{x}} \\ \hat{\mathbf{d}}_{\boldsymbol{\eta}} \end{bmatrix} = \begin{bmatrix} \mathbf{Q} + \rho\, \mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix}^{-1} \Big[\mathbf{I}_{\tilde{\mathbf{v}}} - \nabla_{\tilde{\mathbf{v}}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta}) \Big]^{-T} \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \begin{bmatrix} (-\frac{\partial \ell}{\partial \mathbf{z}^*})^T \\ 0 \end{bmatrix}. \tag{52}$$

From Equation (24) we have:

$$\mathbf{I}_{\tilde{\mathbf{v}}} - \nabla_{\tilde{\mathbf{v}}} F(\tilde{\mathbf{v}}(\boldsymbol{\theta}), \boldsymbol{\theta}) = \begin{bmatrix} \mathbf{Q} + \rho\, \mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix}^{-1} \begin{bmatrix} -\rho(2D\Pi(\mathbf{v}) - \mathbf{I}_{\mathbf{v}}) & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix}. \tag{53}$$

Simplifying Equation (52) with Equation (53) yields the final expression:

$$\begin{bmatrix} \hat{\mathbf{d}}_{\mathbf{x}} \\ \hat{\mathbf{d}}_{\boldsymbol{\eta}} \end{bmatrix} = \left[\begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \begin{bmatrix} \mathbf{Q} + \rho\, \mathbf{I}_{\mathbf{v}} & \mathbf{A}^T \\ \mathbf{A} & 0 \end{bmatrix} + \begin{bmatrix} -\rho(2D\Pi(\mathbf{v}) - \mathbf{I}_{\mathbf{v}}) & 0 \\ 0 & 0 \end{bmatrix} \right]^{-1} \begin{bmatrix} D\Pi(\mathbf{v}) & 0 \\ 0 & \mathbf{I}_{\boldsymbol{\eta}} \end{bmatrix} \begin{bmatrix} (-\frac{\partial \ell}{\partial \mathbf{z}^*})^T \\ 0 \end{bmatrix}. \tag{54}$$

C Proof of Proposition 3

From the KKT system of equations (20) we have:

$$\mathbf{G}^T \operatorname{diag}(\tilde{\boldsymbol{\lambda}}^*) \hat{\mathbf{d}}_{\boldsymbol{\lambda}} = \operatorname{diag}(\rho\, \boldsymbol{\mu}^*) \hat{\mathbf{d}}_{\boldsymbol{\lambda}} = \Big(-\Big(\frac{\partial \ell}{\partial\, \mathbf{z}^*} \Big)^T - \mathbf{Q}\, \hat{\mathbf{d}}_{\mathbf{x}} - \mathbf{A}^T\, \hat{\mathbf{d}}_{\boldsymbol{\eta}} \Big). \tag{55}$$

From Equation (21) it follows that:

$$\frac{\partial \ell}{\partial\, \mathbf{l}} = 0 \iff \boldsymbol{\lambda}^*_- > 0 \quad \text{and} \quad \frac{\partial \ell}{\partial\, \mathbf{u}} = 0 \iff \boldsymbol{\lambda}^*_+ > 0, \tag{56}$$

and therefore Equation (55) uniquely determines the relevant non-zero gradients. Let $\tilde{\boldsymbol{\mu}}^*$ be as defined by Equation (29), then it follows that:

$$\hat{\mathbf{d}}_{\boldsymbol{\lambda}} = \operatorname{diag}(\rho \tilde{\boldsymbol{\mu}}^*)^{-1} \Big(-\Big(\frac{\partial \ell}{\partial\, \mathbf{z}^*} \Big)^T - \mathbf{Q}\, \hat{\mathbf{d}}_{\mathbf{x}} - \mathbf{A}^T\, \hat{\mathbf{d}}_{\boldsymbol{\eta}} \Big). \tag{57}$$

Substituting $\hat{\mathbf{d}}_{\boldsymbol{\lambda}}$ into Equation (21) gives the desired gradients.

D Data Summary

GICS Sector	Stock Symbols							
Communication Services	CBB	CMCSA	DIS	FOX	IPG	LUMN	MDP	NYT
	T	VOD	VZ					
Consumer Discretionary	BBY	CBRL	CCL	F	GPC	GPS	GT	HAS
	HD	HOG	HRB	JWN	LB	LEG	LEN	LOW
	MCD	NKE	NVR	NWL	PHM	PVH	ROST	TGT
	TJX	VFC	WHR	WWW				
Consumer Staples	ADM	ALCO	CAG	CASY	CHD	CL	CLX	COST
	CPB	FLO	GIS	HSY	K	KMB	KO	KR
	MO	PEP	PG	SYY	TAP	TR	TSN	UVV
	WBA	WMK	WMT					
Energy	AE	APA	BKR	BP	COP	CVX	EOG	HAL
	HES	MRO	OKE	OXY	SLB	VLO	WMB	XOM
Financials	AFG	AFL	AIG	AJG	AON	AXP	BAC	BEN
	BK	BXS	C	GL	JPM	L	LNC	MMC
	PGR	PNC	RJF	SCHW	STT	TROW	TRV	UNM
	USB	WFC	WRB	WTM				
Health Care	ABMD	ABT	AMGN	BAX	BDX	BIO	BMY	CAH
	CI	COO	CVS	DHR	HUM	JNJ	LLY	MDT
	MRK	OMI	PFE	PKI	SYK	TFX	TMO	VTRS
	WST							
Industrials	ABM	AIR	ALK	AME	AOS	BA	CAT	CMI
	CSL	CSX	DE	DOV	EFX	EMR	ETN	FDX
	GD	GE	GWW	HON	IEX	ITW	JCI	KSU
	LMT	LUV	MAS	MMM	NOC	NPK	NSC	PCA
	RPH	PNR	ROK	ROL	RTX	SNA	SWK	TXT
	UNP							
Information Technology	AAPL	ADBE	ADI	ADP	ADSK	AMAT	AMD	GLW
	HPQ	IBM	INTC	MSFT	MSI	MU	ORCL	ROG
	SWKS	TER	TXN	TYL	WDC	XRX		
Materials	APD	AVY	BLL	CCK	CRS	ECL	FMC	GLT
	IFF	IP	MOS	NEM	NUE	OLN	PPG	SEE
	SHW	SON	VMC					
Real Estate	ALX	FRT	GTY	HST	PEAK	PSA	VNO	WRI
	WY							
Utilities	AEP	ATO	BKH	CMS	CNP	D	DTE	DUK
	ED	EIX	ETR	EVRG	EXC	LNT	NEE	NFG
	NI	NJR	OGE	PEG	PNM	PNW	PPL	SJW
	SO	SWX	UGI	WEC	XEL			

Table 3: U.S. stock data, sorted by GICS Sector. Data provided by Quandl.